DRISHTI · दृष्टि · SIGHT

Drishti
A live terminal for any record, without a screen per product

Any data. Any domain. One grammar.

Ashutosh Sinha

October 2026

Java 21 or later · console on any device

IN THIS DECK

1 Why: a screen is a paragraph
2 How it works: the design
3 The product
4 Editions and lenses
5 Data in, and the lakes
6 Screens, built without code
7 BI on the same data
8 Risk, computed on demand
9 Security, drawn as flows
10 Evidence, by the numbers
11 Operating it
12 Where it is going
A Appendix: reference
Related documents

TL;DR

Executive summary

What is a live entity terminal?	One dense, keyboard-driven, ticking screen for any single record (a swap, a netting set, a fund, a company, an economy) linked to everything it touches, with its provenance shown.
What is Drishti?	A Java 21 server and a web console that turn any record into that screen, with BI on the same data and masks, and risk computed on demand: VaR and exposure for any scope, from the bank to one trade.
Why does it matter?	Every entity kind traditionally gets a bespoke screen, written once and rotting; every risk figure a stored number nobody can re-derive. Drishti makes a screen a paragraph and a figure a computation.
What can it do today?	11 packs, 219 Sutras, 106 commands with data contracts, 19 source plugins; three editions; VaR by any hierarchy, sharded across servers, on Delta or Iceberg, equal to the cent; exposure; 35.7M trades served.
Where is it going?	Planned: 10-day and stressed VaR, Arrow to the browser, a client-side renderer. Parked by the product owner: counterparty, balance-sheet and regulatory measures, then AI and GenUI.

Why: a screen is a paragraph

01

THE CATEGORY

What is a live entity terminal?

- **One record, one dense screen**
  A swap, an FX trade, a netting set, a ticket or a gene variant, with its headline figures in a strip and its detail in up to twenty-two kinds of panel.

- **Live, not refreshed**
  When the source changes, the open screen changes: only the changed figures are sent, measured at p99 11 ms from tick to frame.

- **Linked**
  Every id on the screen is a link: a trade's counterparty, its netting set, its curve — one keystroke away, across systems.

- **Explained**
  Every view says where its data came from, how fresh it is, and why it is laid out the way it is.

> **Key insight** **The terminal is the standard for dense, live screens — Drishti makes one for any record you have, not only the vendor's.**

THE PROBLEM

Every entity kind gets a bespoke screen — and the screen rots

A screen per product, in the documents' own words

- **Hundreds of entity kinds**
  “Front-office, risk and operations staff look at hundreds of entity kinds.”
- **Written once, rotting for ever**
  “Each one traditionally gets a bespoke screen, written once and rotting forever.”
- **Data outruns screens**
  “The data changes shape faster than screens can be rebuilt.”

And what people do in the meantime

- **Tab between systems**
  “A trade lives in one system, its netting set in another, its curve in a third … the desk tabs between them.”
- **Wait for a screen**
  “Every new product type waits for a screen.”
- **Read raw JSON**
  “A new feed or product has no screen at all, so on day one people read raw JSON or wait.”

Key insight **Nobody here is short of data — they are short of a screen for it that is dense, live and linked.**

THE SHAPE OF THE FIX

From a coded screen per product — to one grammar and inference

Your systems

Trades, curves, tickets, variants

→

A screen per kind

Coded by a developer, per product

→

A backlog

New products wait for a screen

→

Drift

The data changes; the screen does not

Your systems

Read where they are, through plugins

→

DRISHTI

Sutra ⊕ inference

A short YAML Sutra describes the view; scored, explainable rules read the data's shape and fill the rest

→

A live view

Dense, linked, explained — day one

Key insight A brand-new product with no Sutra still renders a usable view on day one; a Sutra makes it beautiful.

Six goals the design is held to

1

One grammar

Every view is a Sutra in Rachana, or inferred; no screen code.

2

Any source

Sources plug in through a small, Spring-free interface.

3

Linked

Every id on a screen is navigable, across systems.

4

Live

Changes reach the open view; p99 is measured and shown in the top bar.

5

Provenance

Every view shows how it was built and where its data came from.

6

Keyboard first

<MNEMONIC> <ID> <GO>, F-keys for panels, F9 for raw JSON.

Key insight Six goals, each one testable — the p99 is not a claim on a slide, it is in the top bar of every view.

Each family does part of the job, and is good at it

Family	Examples	Good at	What it leaves you
Market data terminals	Bloomberg Terminal, LSEG Workspace, FactSet	Dense, keyboard-driven, live screens	The vendor's screens, for the vendor's data
Trading and risk platforms	Murex, Calypso, in-house systems	Deep screens for every product they book	A coded screen per product type, for that platform's data
Low-code internal tools	Retool, Appsmith, Power Apps	Internal screens assembled quickly	A person still assembles each screen
BI and dashboards	Tableau, Power BI, Grafana	Aggregates and charts	Weaker at one entity, live, with its links

Key insight **The screens that are dense and live are coded per product; the screens that are quick to make are not terminals.**

BENEFITS

What a grammar of screens buys

Day one, every product	Inference shows a new feed at once; nobody reads raw JSON.
Screens as reviewed data	A Sutra is versioned, tested, diffed and approved like code — without being code.
One place to look	A trade, its curve and its netting set from three systems on one linked screen.
Trust in the number	Provenance and freshness on every view; masks applied for every reader.
Anywhere it is needed	The console on any device, the API, the CLI, and the same view inside other apps.

Key insight **The cost of a new screen falls from a project to a paragraph — and to nothing at all on day one.**

How it works: the design

02

Eight primitives carry the whole model

Primitive	What it is	Where you meet it
Pack	A folder for one domain: kinds, mnemonics, links, connectors, Sutras, samples, help text; packs inherit	config/packs/ — 11 shipped, e.g. trading, market-risk, funds
Sutra	A short, versioned YAML description of one view, in the Rachana grammar	219 Sutras in the shipped packs; fragments from the UI kit
Match	kind + where (a closed expression) + priority; the highest match wins	match: { kind: trade, where: …, priority: 10 }
Strip	Up to eight headline figures across the top of a view	Source, side, notional, status …
Panel	One of 22 kinds on a 12-column grid, each with an F-key	table, ladder, pivot, surface, map …
Inference	Explainable rules that read the data's shape and lay out what no Sutra covers	infer: true; 'completed by inference'
ViewModel	Plain JSON the server returns and the browser draws	GET /api/v1/views/{kind}/{id}
Frame	A patch of what changed, sent over one live channel per browser	Server-sent events

ARCHITECTURE

From your systems to a view, in one direction

Stores and lakes

Delta, JDBC, Aerospike, Cassandra, MongoDB, Redis, Iceberg, DuckDB

Streams and queues

Kafka, ActiveMQ, RabbitMQ

Files and services

JSON Lines, CSV (deprecated), S3, REST

Public market data

SEC EDGAR, GLEIF, N-PORT, NY Fed, ECB, FRED

DRISHTI SERVER · JAVA 21 · ONE OF A CLUSTER

Fetch, match, infer, link, bind

Virtual threads fetch in parallel; the engine matches a Sutra, merges inference, links ids and binds values; BI answers on DuckDB; VaR and exposure engines compute any scope — 19 source plugins

SAME SERVER

Identity, packs, audit, Drishti*net

Users, roles and masks; audit; a signed pack registry; commands switched off for everyone; peers by gossip on their own port, HTTP/2 between servers

Console

Web terminal, any device

REST API

/api/v1, OpenAPI at /api/docs

CLI

tools/drishti.py, 22 groups

Other apps

<drishti-view>, <drishti-report>

SEVEN STAGES

One command, seven stages, one view

1

Command

TRD MX-20000001 <GO>, or plain words

2

Fetch

Sources in parallel on virtual threads

3

Classify

Which kind is this record?

4

Fingerprint

The data's shape, hashed

5

Layout

Sutra ⊕ inference, cached by shape

6

Bind

Values into panels, in parallel

7

Link

Every id navigable, time-boxed

0.6 ms
Server work for a warm view

3.0 ms
Warm view over HTTP, p99 (gate: 50 ms)

7.2 ms
Structured search over 750 trades

Key insight **The expensive part — deciding the layout — happens once per shape, not once per request.**

A swap's whole screen, in one short file

murex/irs-fixfloat.v1.sutra.yaml (trimmed)

```
rachana: 1
sutra: irs-fixfloat
doc: { summary: "Exchanges fixed for floating …" }
match: { kind: trade, priority: 10, where:
  "$.SOURCE_SYSTEM == 'MUREX' &&
   $.PRODUCT_TYPE == 'IRS_FIXFLOAT'" }
title: { pill: "Rates · Interest rate swap",
         id: $.TRADE_ID }
strip:
  - { label: Source, bind: $.SOURCE_SYSTEM }
  - { label: Maturity, bind: $.MATURITY_DATE }
panels:
  - { use: trading/deal-ticket@1 }
  - { use: trading/legs@1, with: { rate-fmt: pct4 } }
  - { use: data-provenance@1 }
  - { use: risk-links@1 }
```

- **When it applies**
  A MUREX trade whose PRODUCT_TYPE is IRS_FIXFLOAT; the highest priority match wins.
- **What it shows**
  A linked header, eight headline figures and fragments: the pack's deal ticket and legs, the UI kit's provenance and risk links; 132 pack files reuse the deal ticket.
- **Documented in itself**
  Every Sutra and fragment carries a doc: block (summary, params, examples) that the designer, the help and AI read.
- **Shipped through review**
  Rachana-EL has no side effects; a bad edit keeps the last good version live; in production a second person approves.

LIVE, END TO END

One subscription per entity, one channel per browser

Source
subscribe()

→

TopicHub
One topic per entity; latest wins

→

ViewStream
One per open view; cached layout

→

PatchDiffer
Old vs new ViewModel

→

Browser
Patches over one SSE channel

2.4 ms
Tick to frame, p50

11.2 ms
Tick to frame, p99 (gate: 40 ms)

50 ms
A topic sends at once if it has not sent within this window

1
Live channel per browser, however many views

Key insight A view does not poll and does not redraw — only the figures that changed travel, and only to the views that show them.

Several servers, one Drishti

Console
Given one server; learns the rest; fails over

→

Any node
Signed calls; your roles checked

→

Capability table
Who renders each kind, who holds its data

→

Forward or fetch
Newest pack served; masked on both nodes

→

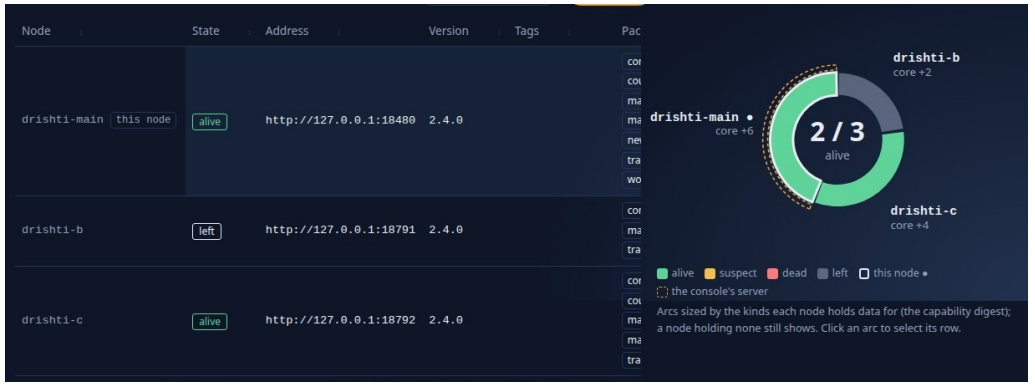
Scatter-gather
A search asks every shard, once

52 ms
Three servers converge after the third starts

0 wrong
of 146,498 answers checked in a 2-hour soak with kills

117 ms
p99 between servers on HTTP/2 (370 ms on HTTP/1.1)

13,542
Signed calls through a key rotation, none refused



Key insight Each node keeps its own packs, connectors and data — the user cannot tell which one answered; an administrator can.

The product

03

JAVA 21 OR LATER · ANY DEVICE

Drishti — overview

11
Packs: core, trading, market data and risk, counterparty risk, funds, US equities, GLEIF, economy, news

219
Sutras shipped in those packs, built from the UI kit

106
Commands, each with a machine-readable data contract

19
Source plugins behind one interface

- **A terminal on any device**

Command line, history ranges, FUNC, DATAMAP, Knowledge and WHAT over production-shaped trades from ENDUR, CALYPSO and MUREX and public data from SEC, GLEIF and N-PORT; phones.

- **Risk computed, never stored**

VaR and exposure for any scope, baskets and pre-deal what-ifs, official end-of-day records, in BI; VaR from scenario PVs in any reporting currency.

- **Build and analyse without code**

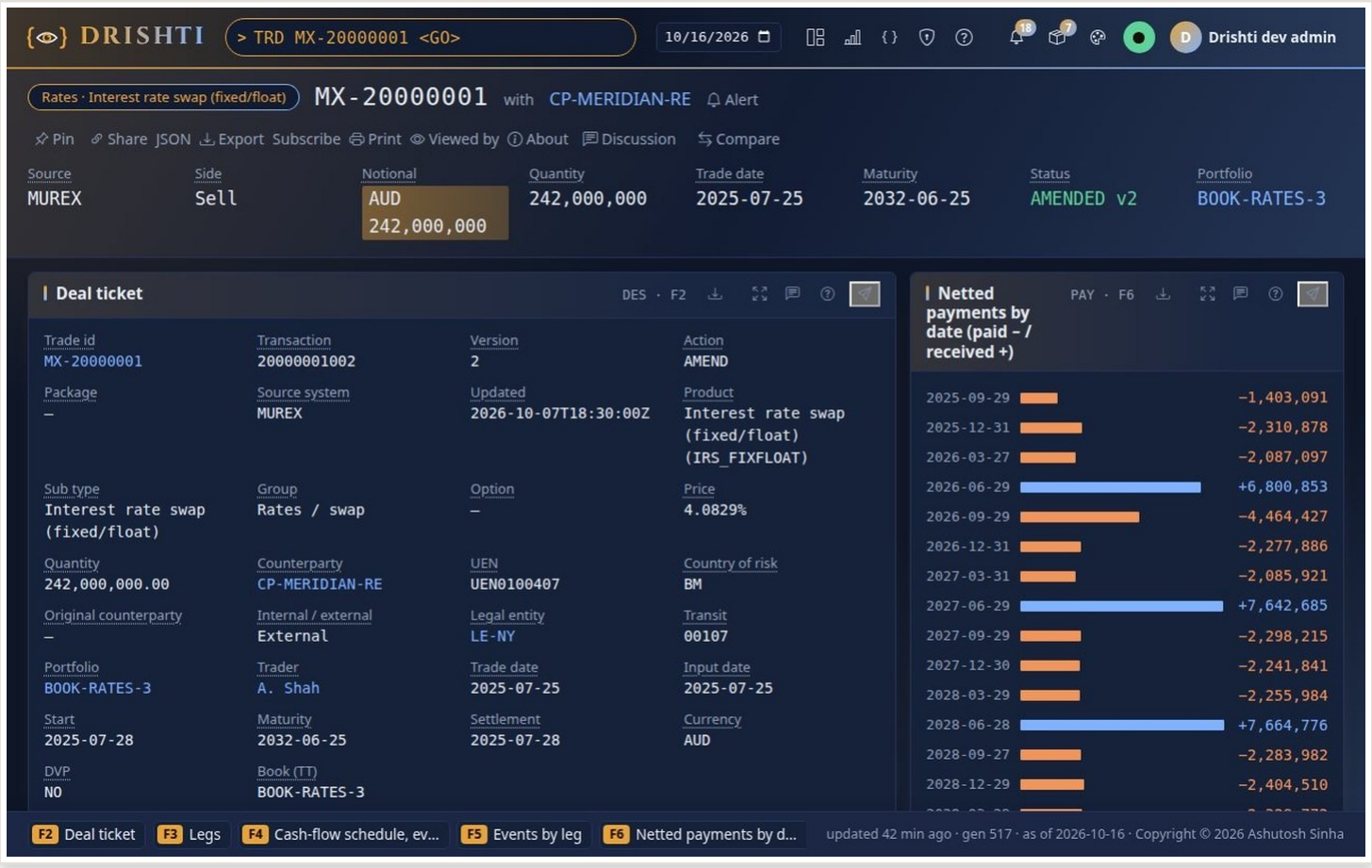
The four-mode Sutra designer and the UI kit; BI Studio with runs, statements and charts of any table; Python data prep; forms that write back under four eyes.

- **Run it, on one server or several**

Pack bundles with rollback, data-load signals, JSON Lines in and out, Delta and Iceberg side by side, a CLI for every step, Drishti*net clusters.

THE VIEW

One trade, linked and explained



A MUREX interest rate swap, UPPER_SNAKE as production data is; captured on the demo lakes (tutorial 1).

- **Strip**
  Source, side, notional, quantity, dates, status and portfolio.
- **Panels by key**
  F2 deal ticket, F3 legs, F4 cash flows, F6 netted payments; Alt+L to rearrange.
- **Linked**
  Counterparty, portfolio, legal entity and trader open in one click.
- **Any day**
  The business date in the top bar; Compare shows what changed between two days.

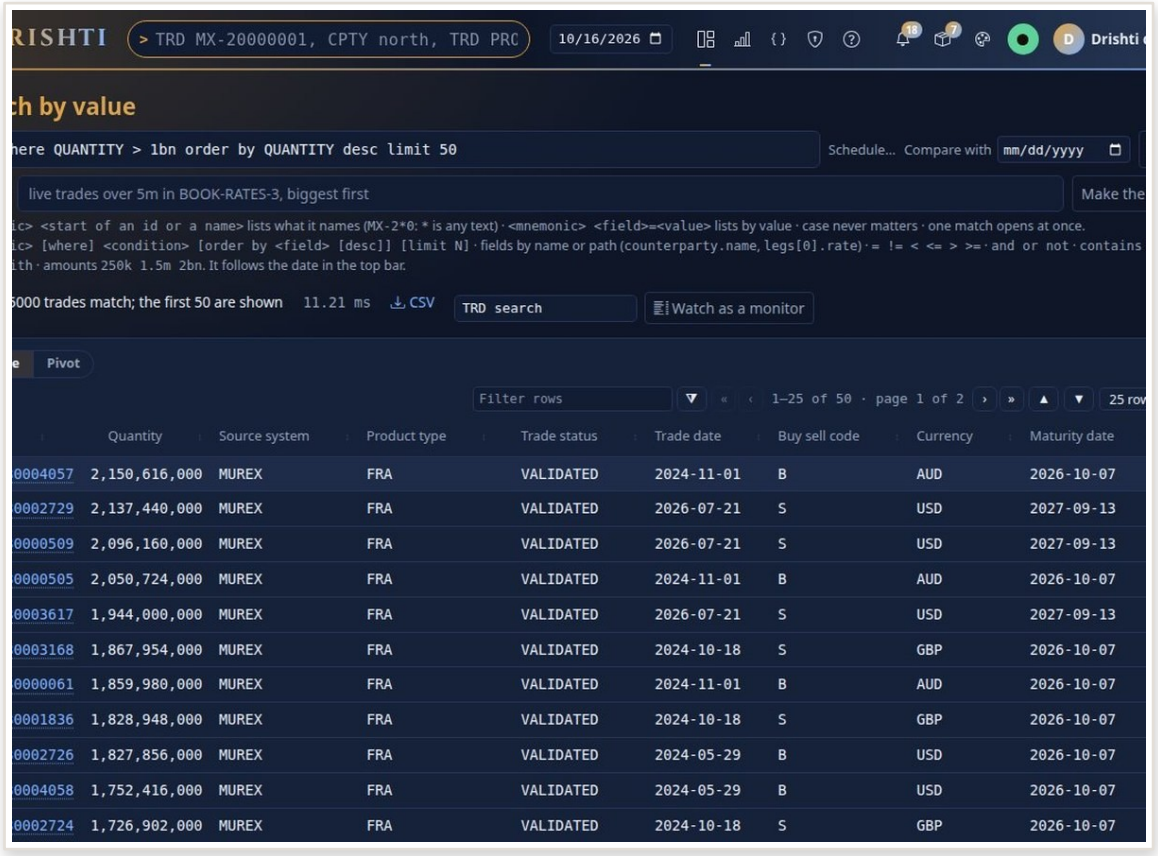
ABOUT THIS PAGE

Every view explains itself, in five layers



- **What you are looking at**
  From the pack's own help text and glossary.
- **Where the data came from**
  Source, health, generation and freshness.
- **Why it looks like this**
  Which Sutra matched, which lost, what inference added.
- **Masked for you**
  Says which fields your role cannot see.

Ask in a grammar or in plain words; pivot what comes back




```
TRD where QUANTITY > 1bn order by QUANTITY desc
amended trades over 5m in BOOK-RATES-3
FORM trading <GO>
```

134 ms

Search all 1M trades a day, 2,017 matches

29 ms

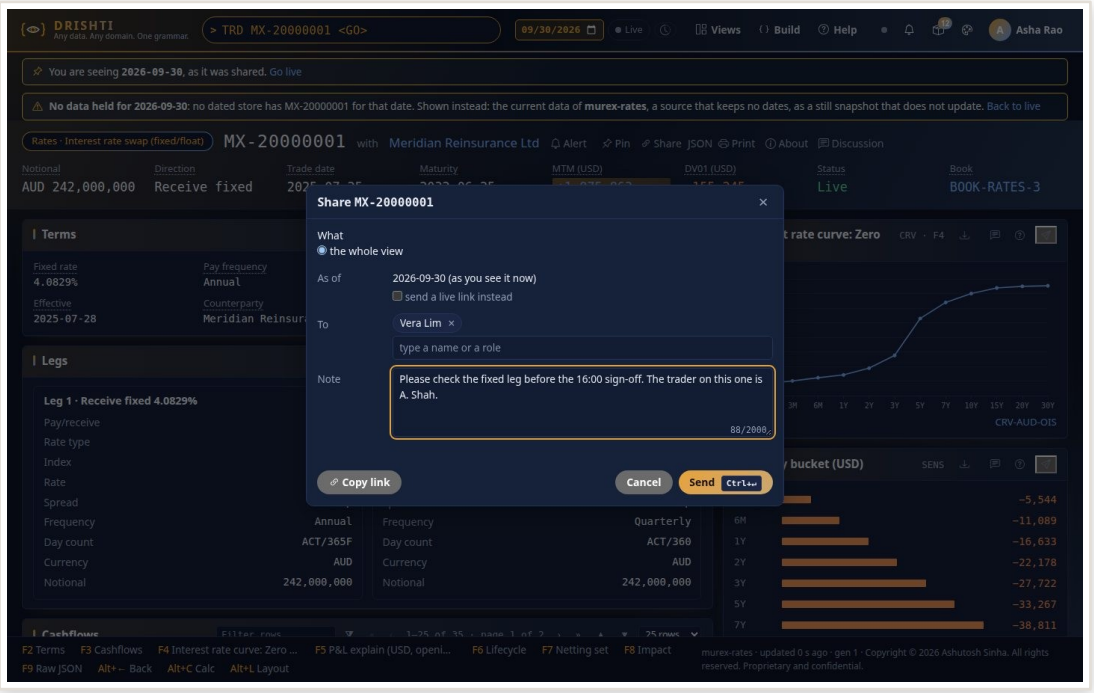
Type-ahead at 1M trades a day

Measured on a 24-core workstation, 1,000,000 trades a day for three days (5.2 GB).

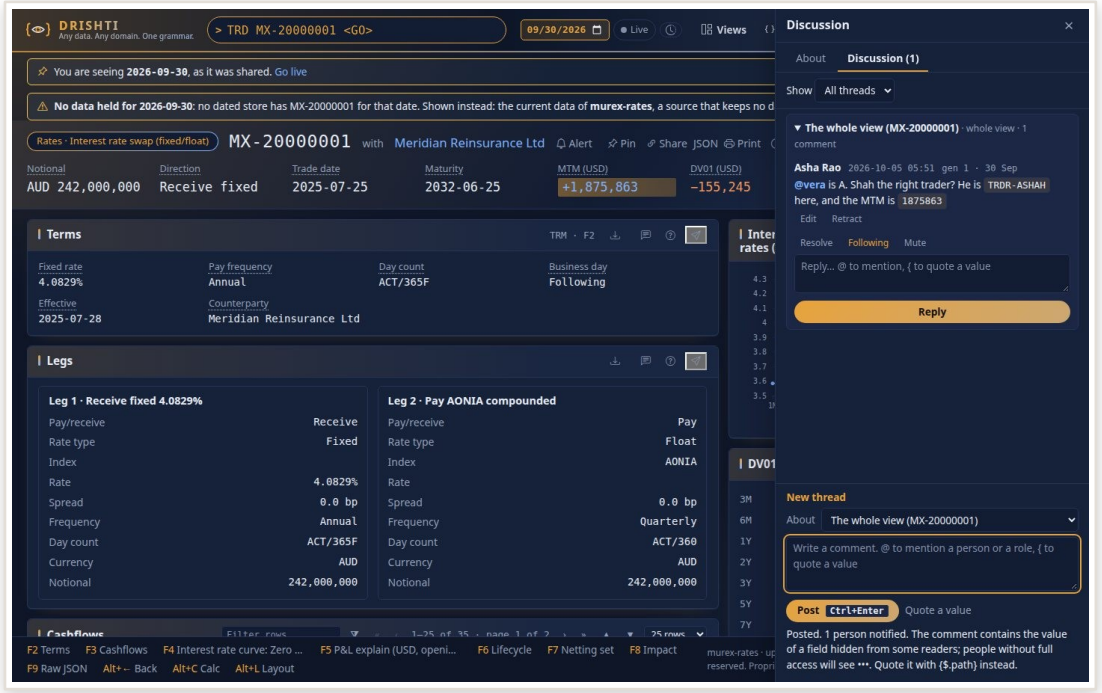
Key insight **Search is a question in the terminal's own grammar — and its answer is a view you can pivot, compare and share.**

COLLABORATION

Share what you see, discuss it, and prove what was said



Share with a note, to people or roles.



A discussion thread with @mentions and quoted fields.

Key insight **Every comment is hash-chained and sealable; retract and hide never delete — evidence, not just conversation.**

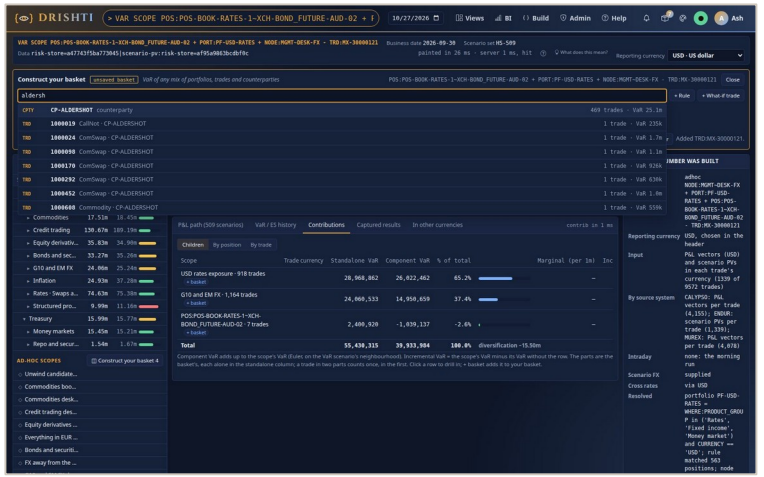
Explain the data, then let anyone build the portfolio they mean

Knowledge pages

- Market risk
- Counterparty credit risk
- Trades and products
- Entities and ownership
- Market data
- Data and contracts
- Economies
- Companies, funds and ETFs

Each with the plain idea, how the records connect, a worked example on the demo data and the questions to ask.

`WHAT <term>` 162 terms; 'What does this mean?' beside the VaR and exposure explorers, USEQ, FUND, HELD and TRACK.



Construct your basket

Pick portfolios, desks, books, legal entities, counterparties, positions, trades or a rule as you type; see VaR, ES and diversification live; save it. Alt+B.

The pre-deal what-if

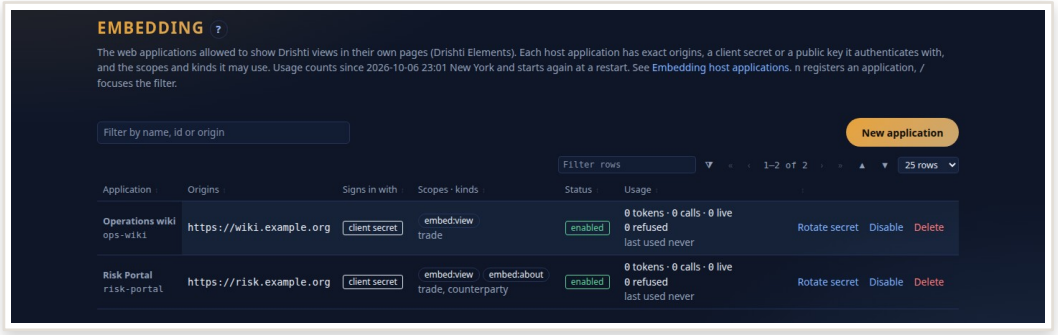
A hypothetical trade — a booked one copied and scaled, −1 its unwind — shows VaR before and after in the basket; the exposure explorer has it on the counterparty and the netting set. Exact methods run on the compute workers; trade pages link to the trade's P&L vector, VaR and exposure, or say why not.

Key insight An id that matches nothing, or a trade that is not live, is said in words — never shown as a VaR of 0.

The same live view, inside other web applications

```
<script type="module" src=
  "/embed/v1/elements/1.4.3/drishti-elements.js">
</script>

<drishti-view server="https://drishti.example"
  kind="trade" entity="MX-20000001"></drishti-view>
```



Admin → Embedding: host apps, origins, scopes and usage.

HTML

Over the wire: the console's own templates, sanitised, in a Shadow DOM

1 stream

Shared by every element on the host page

@container

Responsive to its own box: a sidebar gets the narrow layout

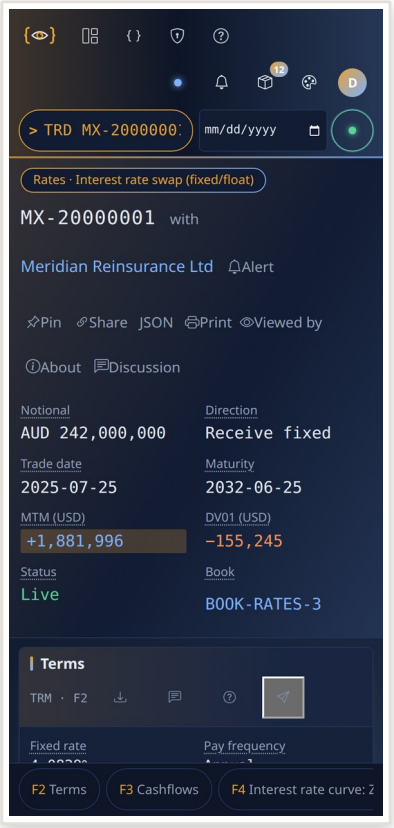
Masked

Always, whatever the user's roles; every call audited

Key insight Other teams get the view, not a copy of it — still live, still masked, still audited.

ON A PHONE

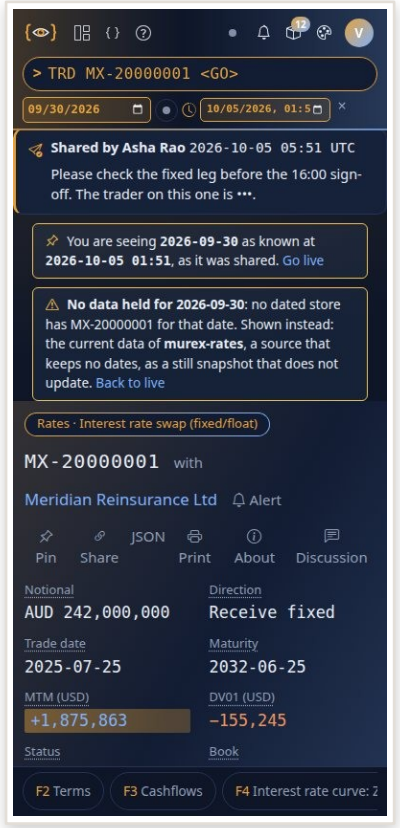
The same terminal in a pocket



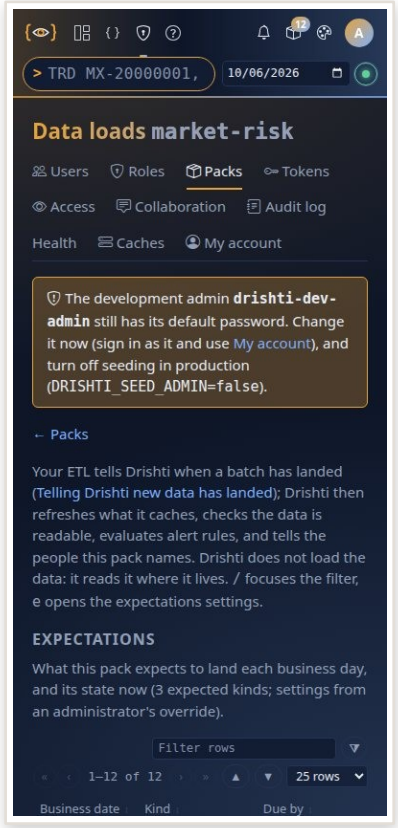
A live trade



About this page



A shared view



Data loads

- **44 px**
  Every control, on touch screens only
- **390 px**
  Every page tested at phone width
- **3 engines**
  Embedded views in Chromium, Firefox, WebKit

Editions and lenses

04

One setting turns Drishti into a focused product

config/editions/risk.yaml (abridged)

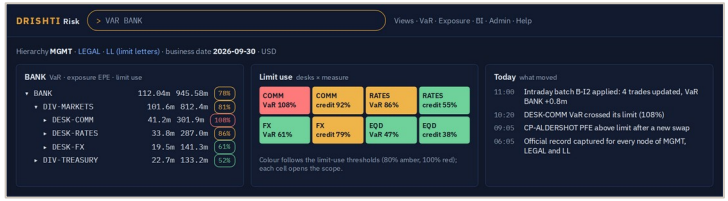
```
edition: risk
title: Drishti Risk
packs: { enabled: [core, market-data, trading,
          counterparty-risk, market-risk],
         allow-only: true }
commands:
  allow: [VRES, VWIF, VOFF, EXP, NSET, …]
  words: { BI: on, FUNC: on, RUPAKA: off }
features: { var: on, exposure: on, designer: off }
bi: { datasets: [market-risk-var-latest, …] }
menus: { views: [terminal, search, var, …] }
home: { landing: desk,
        landings: [desk, command, map] }
branding: { product: Drishti Risk }
```

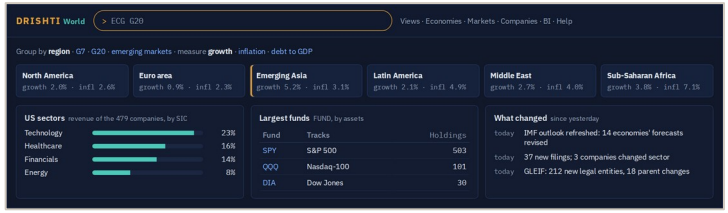
- **One file sets it all**
  Packs (allow-only), commands and words, menus and features, BI datasets and models, Home and its landings, help, Admin pages and branding.
- **DRISHTI_EDITION, on every server**
  drishti.edition: risk names the file; blank is the full Drishti, exactly as without editions. The console learns it from the server.
- **The site still wins, key by key**
  The edition is a layer under the site's configuration: adjust one item without forking it. Switching is configuration and a restart, never code.
- **The engines do not change**
  VaR, exposure, intraday, official records and the what-if run the same; their whole suites run again under the edition.

Key insight A focused product is one more file in config/editions/, not a fork of the code.

The whole instance, or one person's view of it

	Edition	Lens
Set by	configuration and a restart: drishti.edition (DRISHTI_EDITION)	a link, per person: Open Drishti Risk / World / Market
Scope	the whole instance, every member of a cluster	one person's session, kept in their settings
Packs	only the edition's (allow-only)	all stay loaded
Offered by	the administrator	the site's drishti.lenses (DRISHTI_LENSES): risk,world,market
Secures?	refuses other packs; hides commands and pages	no: it hides, it does not secure





The map landing, in Risk and in World: it follows the person from one lens to the other.

Views → Open …
Drishti Risk, World or Market, with each one's description

→

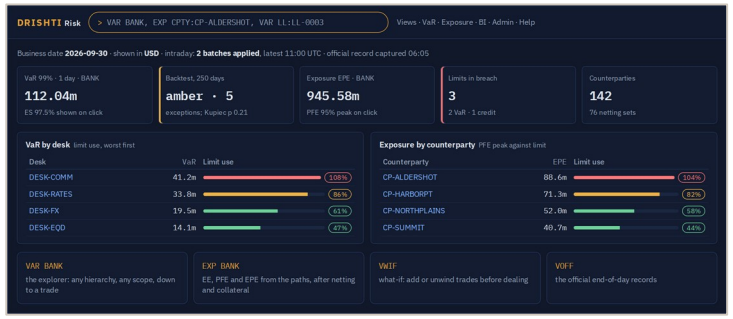
Leave … · Switch to …
inside one lens: leave it, or switch in one step

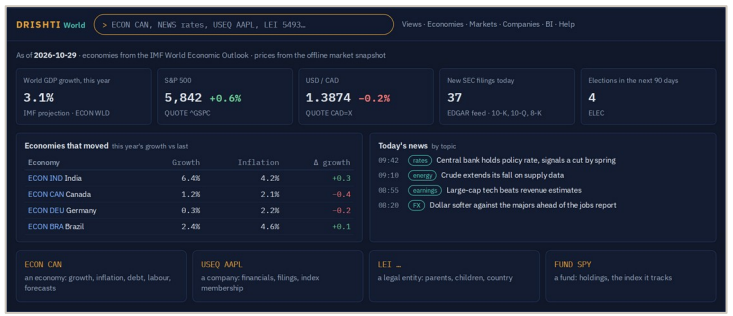
→

Every call
X-Drishti-Lens: commands, FUNC, type-ahead and BI follow

Key insight A lens hides; roles, entitlements and masks still decide what anyone may read.

Drishti Risk, World and Market, each with three landings of its own







Drishti Risk

VaR and exposure first

5 packs: core, market-data, trading, counterparty-risk, market-risk; 36 commands

Landings **desk · command · map**

Drishti World

Economies, markets, companies and funds

5 public-data packs: world-economy, news-and-markets, us-equities, gleif, funds; 32 commands

Landings **desk · explorer · map**

Drishti Market

The bank's curves, surfaces and fixings

market-data and core; 22 commands; no trades, no VaR; quality checks

Landings **desk · explorer · quality**

Role hooks

`home.landing-by-role` is read and wins over a person's choice, e.g. risk-manager: map, trader: command. `lens.roles` (who sees a lens) is documented, not wired.

PLANNED

Key insight **Three products from one engine: each new edition was one file and its landings, no engine change.**

Data in, and the lakes

05

One JSON object a line: every generator writes it, every export offers it

Generators
every tool under tools/ writes JSONL; .jsonl.gz where the repository gzips

→

JSON Lines
one object a line, UTF-8; generating and loading are separate steps

→

Load
load_rows.py, lake build: the lake byte for byte the same as before

→

Delta or Iceberg
the lake every connector reads

Readers

- JSON Lines, JSON arrays and per-entity .json still read
- CSV deprecated: one warning a file, named in Admin → Health
- drishti.py convert FILE: CSV, a JSON array, gzip or a folder into JSONL

Exports

- Server: search, SQL, BI runs, /export/jsonl, statements, subscriptions, scheduled reports, VaR and exposure
- Console: every download and menu; CLI: --jsonl, --format jsonl
- CSV, Excel, PDF and PNG all stay

Records

- Official VaR and exposure results as JSONL beside their Delta tables
- Sutra test cases: one cases.jsonl per Sutra, 197 folders converted
- Sample catalogues, staged lake rows, seals: JSONL

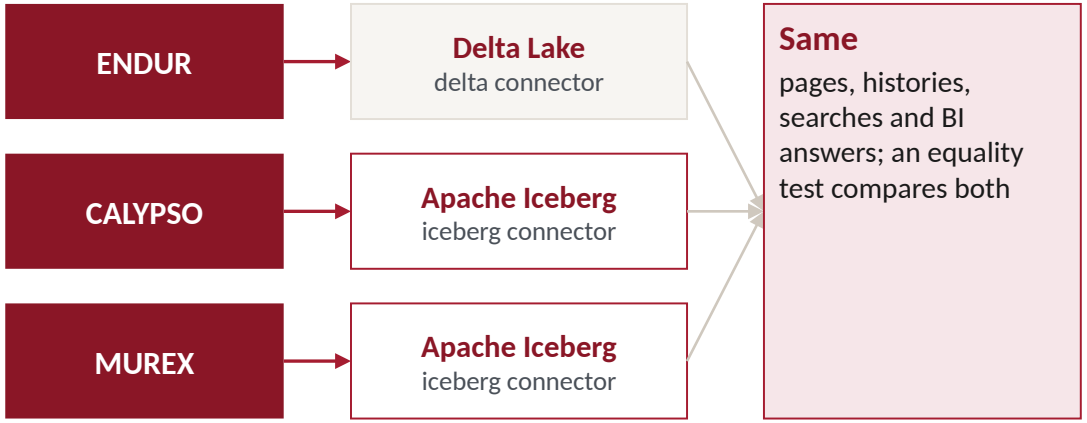
Guarded in the drill: `test_generators_write_jsonl.py` runs the generators and reads every data file; `test_exports_offer_jsonl.py` checks every export entry point.

Key insight A pipeline streams any Drishti input or output a line at a time; CSV, Excel, PDF and PNG stay for people.

Each system's trades in Delta or Iceberg — the same answers

One switch per source system

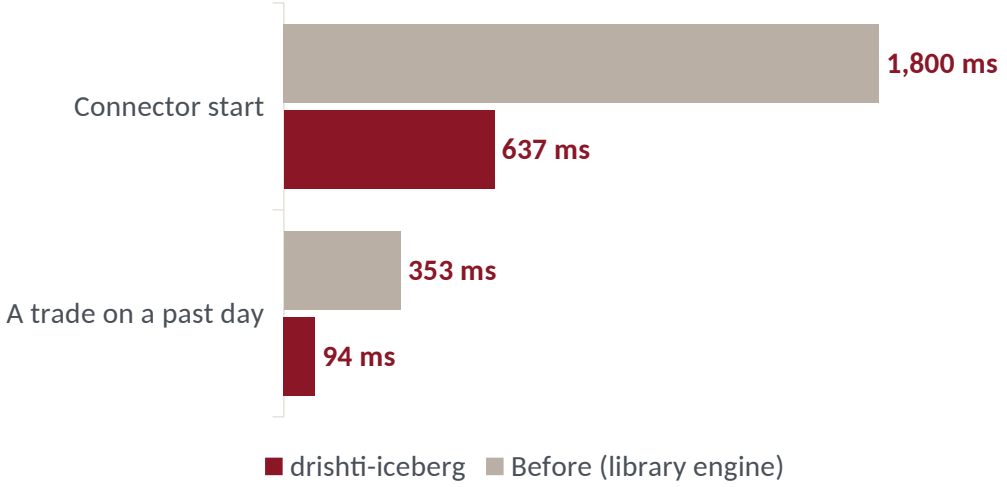
```
build_demo_lakes.py --store endur=delta \
    --store calypso=iceberg --store murex=iceberg
```



The iceberg-external connector also reads tables Drishti did not write: Spark, Flink, Trino, Snowflake, Glue.

drishti-iceberg: Iceberg without Hadoop

Drishti's own engine, the default — so Iceberg runs on Windows



2.8×[1]
faster connector start

3.8×[2]
faster read of a past day

Key insight **The store is an admin's decision per system, not a rewrite: the pages cannot tell which format answered.**

[1] 1,800 ms ÷ 637 ms = 2.8: connector start, library engine against drishti-iceberg, both measured (RELEASE_NOTES, DRISHTI_ICEBERG.md) [2] 353 ms ÷ 94 ms = 3.8: a trade on a past day, both measured

Arrow where it pays; our own formats where they are faster

Used today

Java → Python compute workers
Arrow IPC files, memory-mapped, read and written in slices

BI answers
Arrow or JSON, as the caller asks

The console's Python
pyarrow

Deliberately not

VaR hot files
our own float32 position rows in hierarchy order, memory-mapped, float64 node totals: about 70 GB/s[1] summing on 24 threads, near the memory limit

The lake
Parquet in Delta or Iceberg: Arrow's partner on disk

Shard partials
raw vectors of doubles between servers; Arrow would add framing, no gain

Next

PLANNED

Arrow to the browser
server → console → browser for large tables and charts, read with Arrow JS

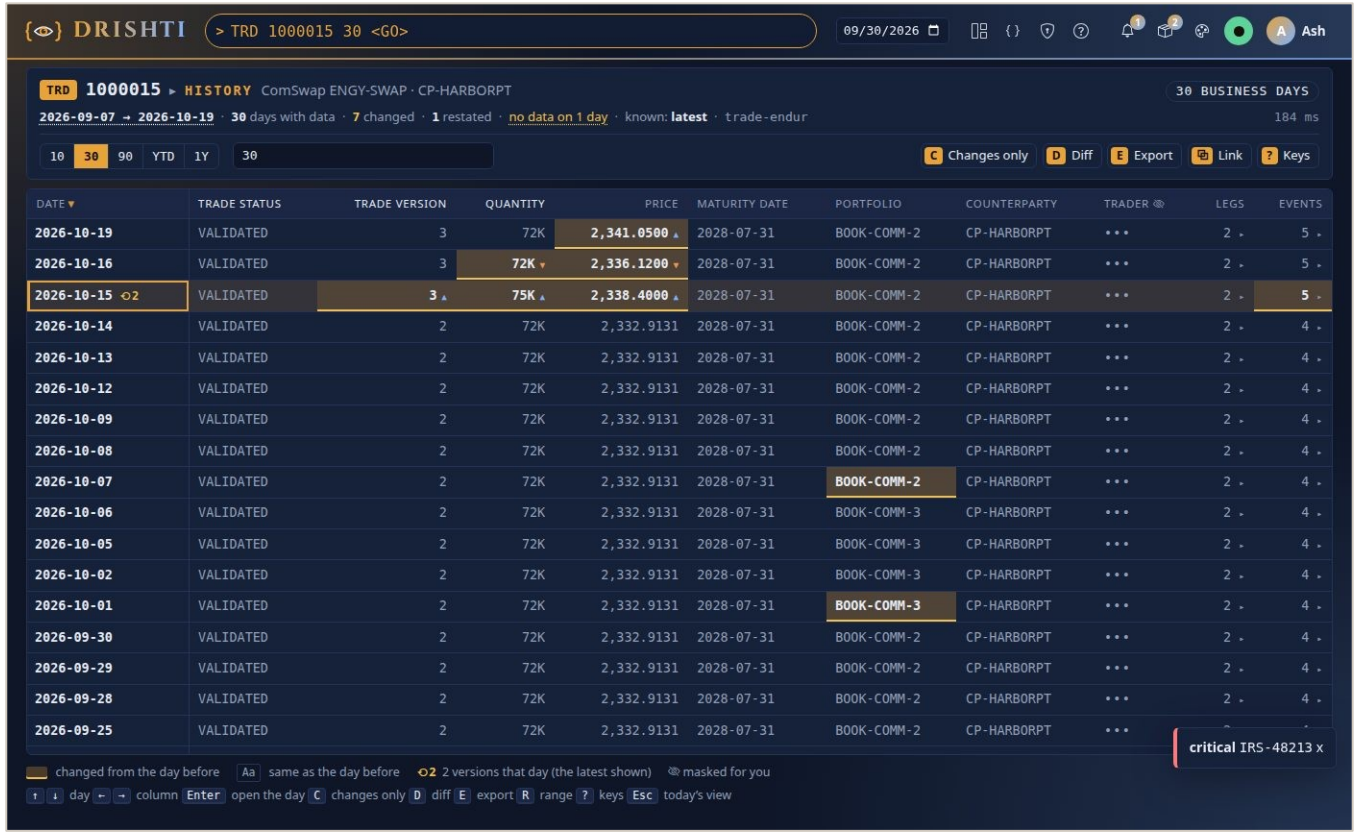
Where
BI grids, 509-point P&L paths, big search results; entity pages stay JSON

Measured before and after
payload size, parse time, click to painted on the largest grids

Key insight A format is chosen per path by measurement: Arrow where data crosses a process, not where vectors are summed.

[1] 1,000,000 trades × 509 × 8 B = 4.07 GB summed in 58 ms (VarScaleBenchmark, measured, VAR_GUIDE.md 27.2) = 70 GB/s

Type a range after an id and see the entity over time



TRD 1000015 30: one row per business day, changed cells marked ▲/▼, restated days flagged (tutorial 21).

One grammar, any range

```
TRD 1000015 30
TRD 1000015 3M DIFF
TRD 1000015 FROM 2026-09-01 TO 2026-09-30
trade 1000015 last month
```

13 ms
30 days of one trade among 10,000, cold

< 1 ms
the same, warm

0.3 ms
from the history cache

Generic: any kind with dated data, no code per kind; fast paths on Delta, Iceberg, Redis, PostgreSQL, DuckDB, MongoDB, Aerospike and Cassandra; masks hold on every day's row.

Key insight Across a cluster, a node reads an entity's days from the peer that holds them in one call, masked on both.

US companies, legal entities and funds — linked, offline, refreshed by script

us-equities

586

US companies from SEC EDGAR, Nasdaq and GLEIF: financials, filings, index members

`USEQ · IDX · SIC · EDGAR`

LEI

gleif

9,512

legal entities in 540 ownership groups; groups computed from DIRECT_PARENT

`LEI · LEIG · LEIC`

LEI

funds

200

largest US mutual funds and ETFs from N-PORT, N-CEN and prospectuses; 5.6 MB gzipped

`FUND · HELD · TRACK · FUNDS`

Refresh script
`tools/publicdata/<pack>.py .sh and .ps1, status, dry-run`

→

Committed snapshot
`data/testdata/<domain>/<kind>/*.jsonl.gz`

→

Demo lakes
`build_demo_lakes → Delta`

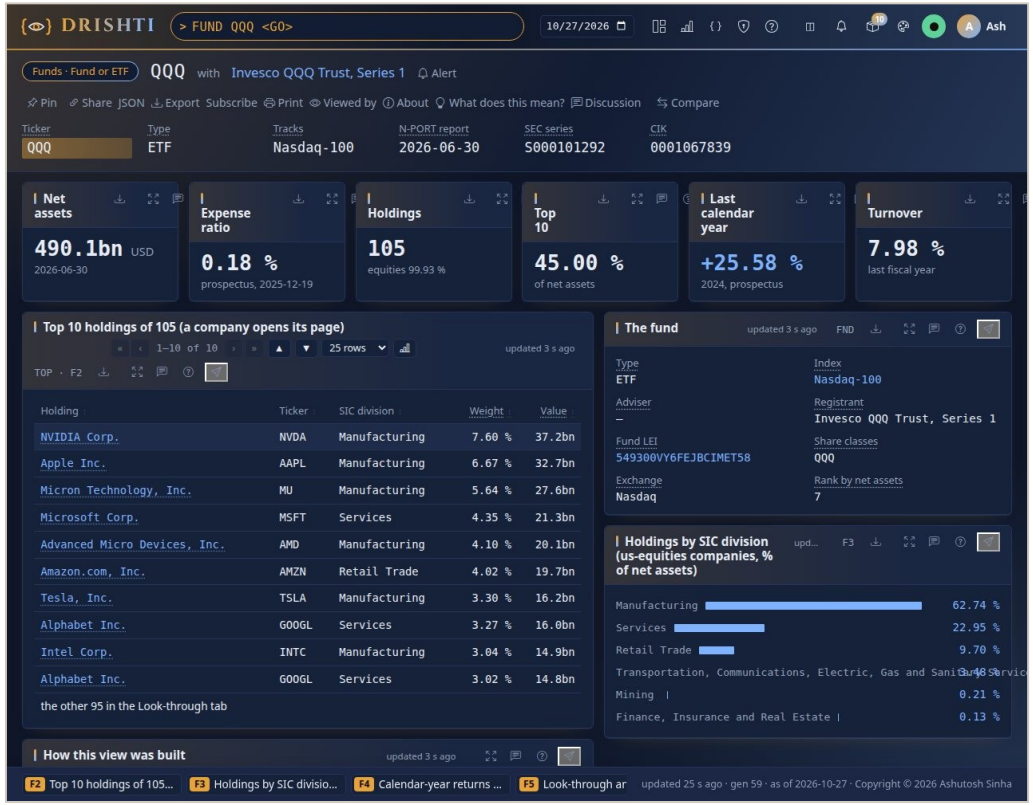
→

A connector per pack
`served offline, like any source`

External data, one page: every source with its URL, licence and use; IMF, Yahoo and Google News marked internal only and refused in the repository; sectors are SIC divisions because GICS is proprietary — the pages say so.

Key insight **Public data arrives the way a bank's data does: a contract per kind, a lake, a connector — and no network at demo time.**

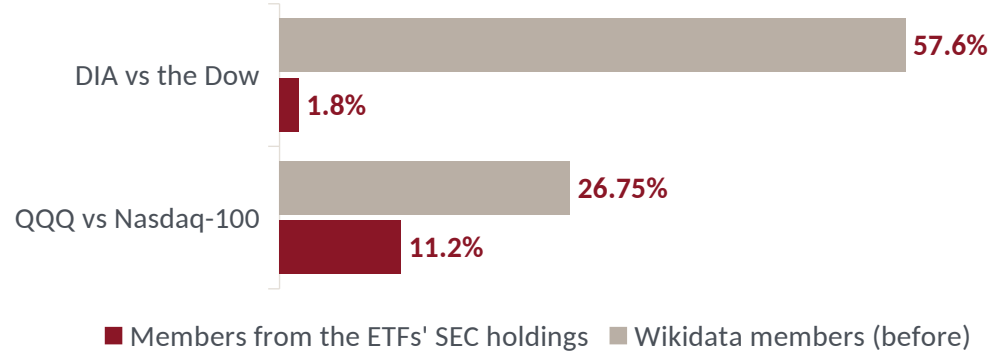
Index members from what the tracking ETFs report to the SEC



FUND QQQ: fact sheet, top holdings, look-through by SIC division (tutorial 28).

Earlier the S&P 500, Nasdaq-100 and Dow members came from Wikidata's lists, which lacked Exxon Mobil, BlackRock, NVIDIA in the Nasdaq-100 and about 100 more.

Active share against the index, before and after



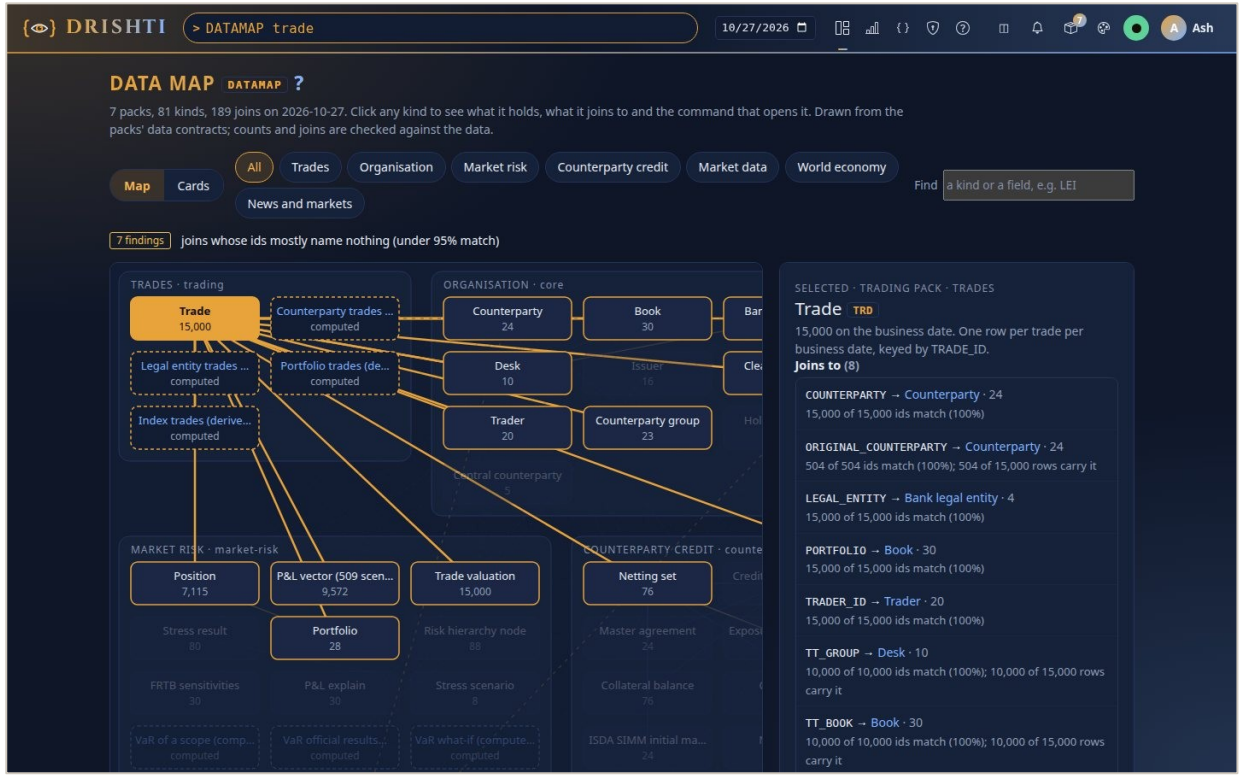
503 → 586
companies in the pack

98 of 99
listed Nasdaq-100 members held by QQQ

162
WHAT terms, 23 new for funds

Key insight The Dow is weighed by price and the Nasdaq-100 capped, as their methods do — the active shares fell to what an investor expects.

A map of all your data that draws itself — and one true surprise a day



DATAMAP trade: domains as clusters, joins as lines, each join's match rate checked on the business date.

- **Nothing hand-drawn**
  Kinds from the enabled packs, joins from the data contracts, counts from the lake; a new pack appears with no setup.
- **Every join checked**
  How many ids match; it found 11 joins that do not hold (e.g. a jurisdiction code where an id is expected).

HOME · THE FACT OF THE DAY

One surprising, true sentence from your data, computed as you, on your business date.

Seven ship: a book over its VaR limit while its desk is within · a desk that lowers VaR · posted collateral raising exposure · VaR against peak PFE · the most-held company · two managers holding one portfolio · a fund's weights above its net assets

Key insight Each pack declares its facts; only those that hold today are shown, with 'Why does that happen?' and the command.

One contract per command, and everything else is generated from it

config/packs/<pack>/contracts/<kind>.contract.yaml

```
kind · mnemonic · grain · key
columns: name, type, unit,
  meaning, source, example
delivery · validation
entitlements · retention
kind of data: fundamental,
  engine input, own, computed
```

drishti.py contract list, show, check, docs, bootstrap; kept current by tests

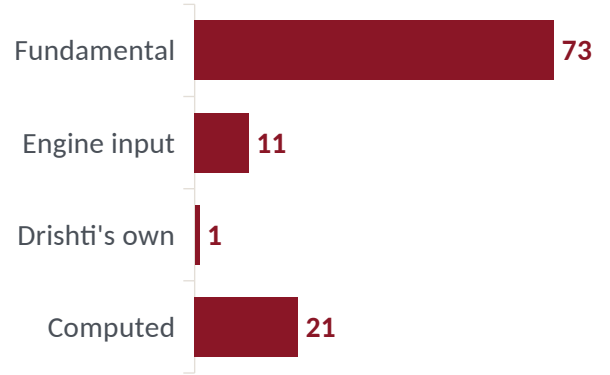
Generated pages docs/contracts and the COMMAND_CATALOG: every command, what it does, an example

Every pack's guide ends with its contracts and where each kind's data comes from

DATAMAP joins between kinds drawn from the contracts and checked against the data

FUNC Views → Functions: every command you may use, by pack; FUNC <words> filters

The 106 commands, by kind of data



A computed figure is never stored in a fundamental or input table: a guard test fails the build.

Admin → Commands
per command

Switch off
any command, for everyone, with a reason

→

Stored and gossiped
the Drishti database, every server

→

'Not active' everywhere
page, API, CLI, search, exports, BI, AI: 403 DRS-1016

Key insight **A command's meaning lives in one file; the catalog, the guides, the map and the switches read it.**

CONFIGURATION, NOT CODE

The product's name and the bank's names come from configuration

Branding

`ui.product · drishti.branding.product`

`ui.owner · ui.owner_email`

No template, script or server message writes the product's name; features follow it: <product> SQL, <product> Elements, <product>*net. The landing headline and tagline too.

Guard `test_branding_guard.py · BrandingLiteralTest`

The bank's names

`config/demo/names.yaml`

`lake demo --names FILE --dry-run`

The bank, its short name, the legal entities LE-NY, LE-LDN, LE-FRA, LE-TKY and branches, renamed as the demo loads, in every kind. Defaults change nothing; LEIs and BICs stay; committed test data is never rewritten.

Guard `tools/lake/test_names.py, in the drill`

The VaR tree and method

`DRISHTI_VAR_ROOT_NAME`

`scenario-window · confidence · currencies`

The top of every VaR tree named by a setting (per hierarchy too); the window, confidence and reporting currencies are settings, and every answer carries its labels.

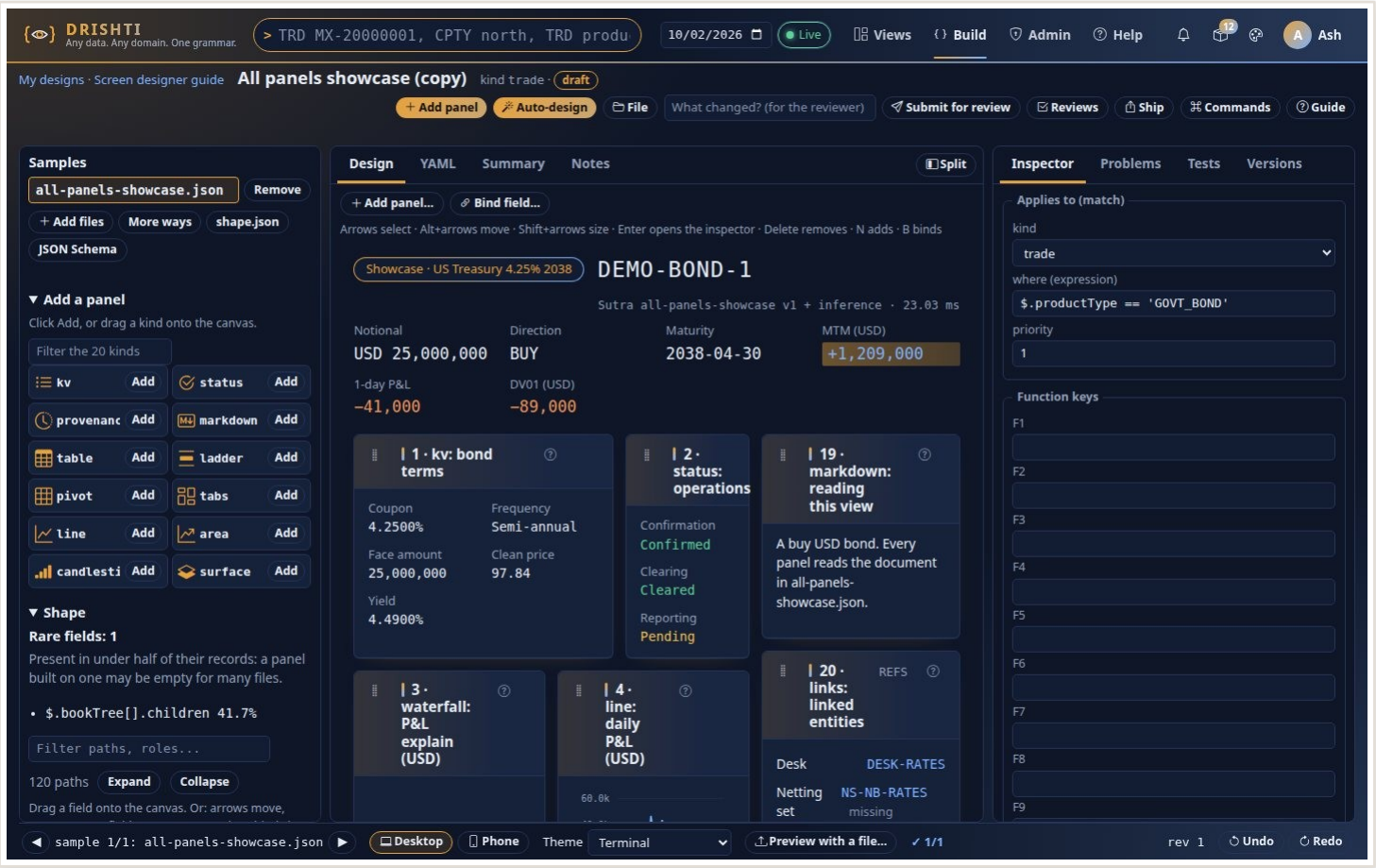
Guard `VarRootNameTest`

Key insight **A bank sees its own names in a demo by editing one file — and no code names the product or the bank.**

Screens, built without code

06

Design a screen without writing code — and ship it through review



- **From data you have**
  Start from JSON files, a schema or stored entities.
- **Drag, or type**
  Palette and inspector, or the YAML tab with completion.
- **Tested before shipped**
  Sutra tests run on sample data in the tests tab.
- **Four eyes**
  An approver reviews the evidence and cannot approve their own.

Screens that respond, built from one UI kit

Events and children

A click on a row opens linked records under it, beside, below or in tabs, from any enabled pack (emits, @panel.event, @parent); masked ids stay hidden.

Every pack screen now uses fragments, each proven to draw the same as before.

Refresh and freshness

refresh: per panel or screen; pause, resume and interval per person; every panel says 'updated N s ago' and names a source that is not answering.

The Drishti UI kit

config/ui-kit: 17 reusable fragments (row-picker@1, field-list@1, cashflow-ladder@1 …), pack fragments, theme tokens and brand; every fragment and Sutra has a doc: docstring

Layout

Named areas on a 12-column grid, splits, tabs, sections, a sticky header, phone order; user-can-arrange per reader.

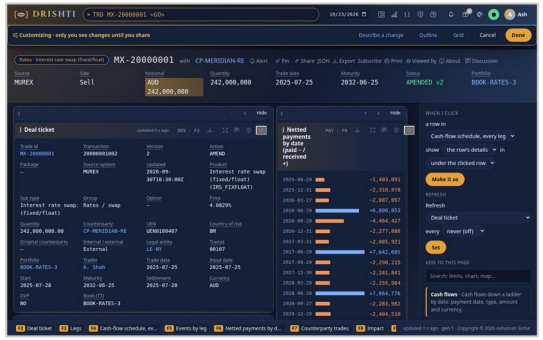
On a cluster, one server per screen; when a server goes away the page says 'your selections were reset' in words.

Forms

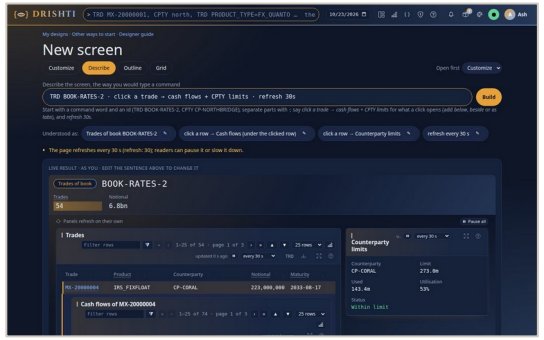
Edit beside a table, in its cells, or New in a guided drawer; linked-record fields, rules and masks; saved through write-back.

Key insight A screen is still a paragraph — now a paragraph of documented, reusable fragments that respond to the user.

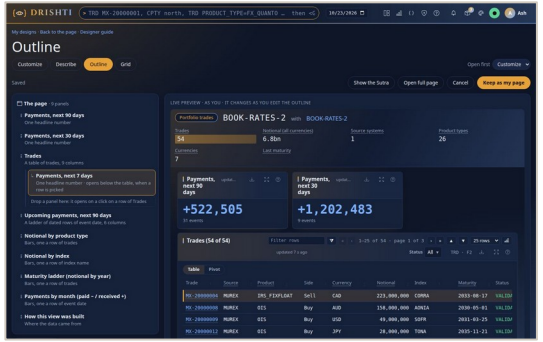
Four modes over one Sutra: customize, describe, outline, grid



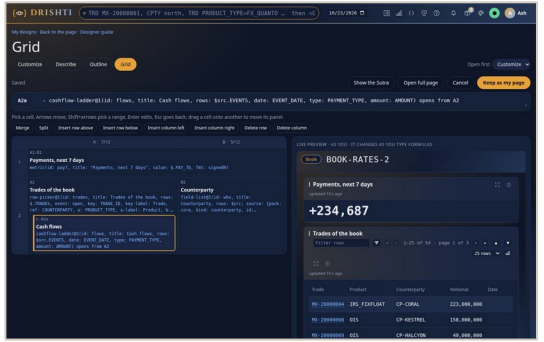
1 **Customize the live page**



2 **Describe a screen in one line**



3 **Outline: the page as a tree**



4 **Grid, with formulas per cell**

A cell is a fragment call

```
B2 = field-list@1(...;
     source: {... id: @A2.open.ref})
```

- **One Sutra, four views**
  Every mode edits the same file; switch at any time.
- **My version of this page**
  Each person keeps a customised copy; the shared page is untouched.
- **Two levels of click-opened cells; off by default**
  ui.designer; tutorial 22 builds a screen with it.

Key insight **Designing a screen in plain words still produces a reviewed, versioned Sutra — not a one-off.**

BI on the same data

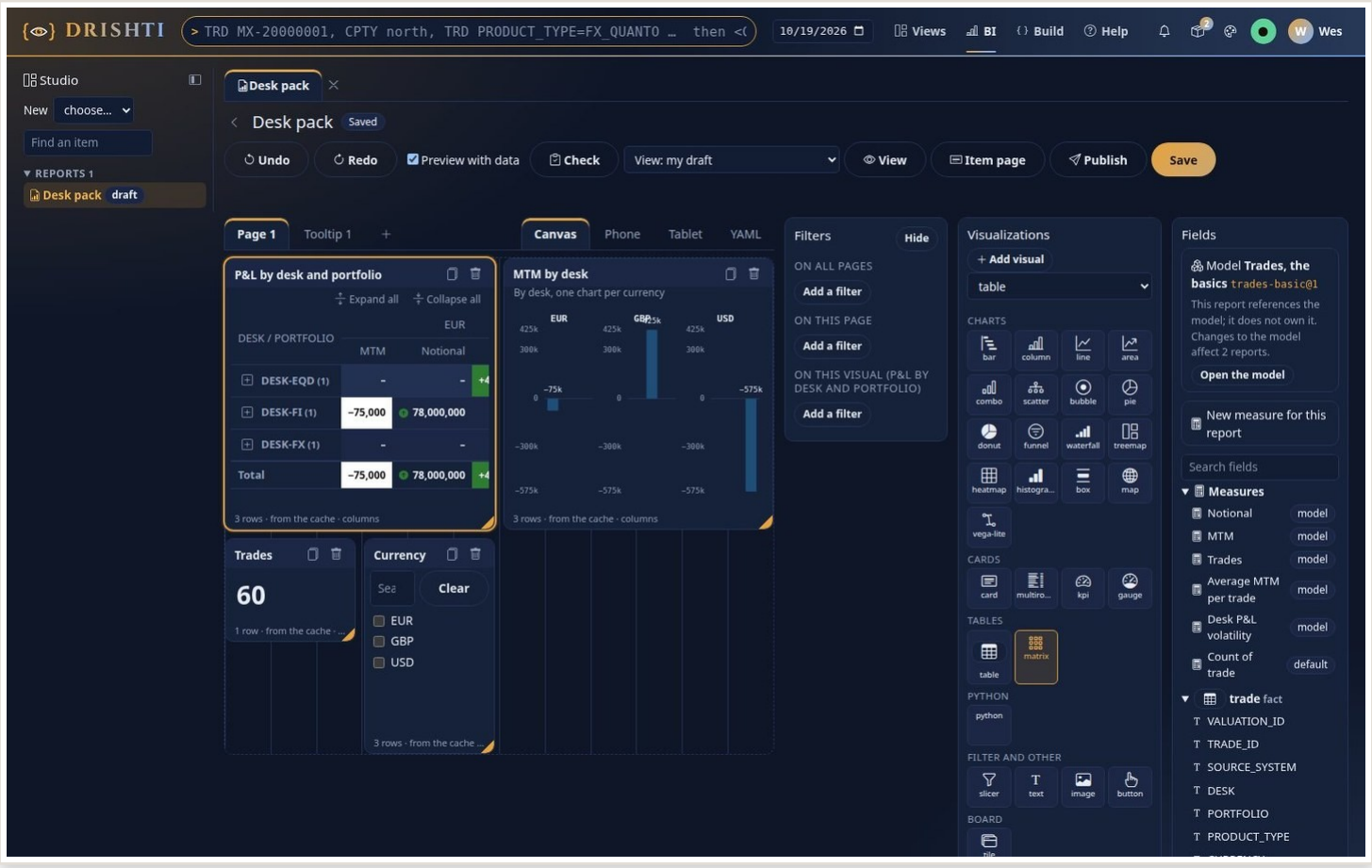
07

Business intelligence on the same data, rules and masks

Models rooted in packs	Datasets and data models built from the packs' kinds, with the relationships the packs suggest.
Three languages, one plan	A Python-like expression language, read-only SQL and Python meet in one plan, your row rules and masks applied.
Servers compute	The Java server and the console's sandbox answer every number; the browser draws; standing queries push live.
Governed	Versions and diffs, four-eyes review, certification, lineage past the connector, bundles with rollback.
Everywhere it is needed	Subscriptions and alerts per reader, PDF, PNG, Excel and CSV, and <drishti-report> in other apps.

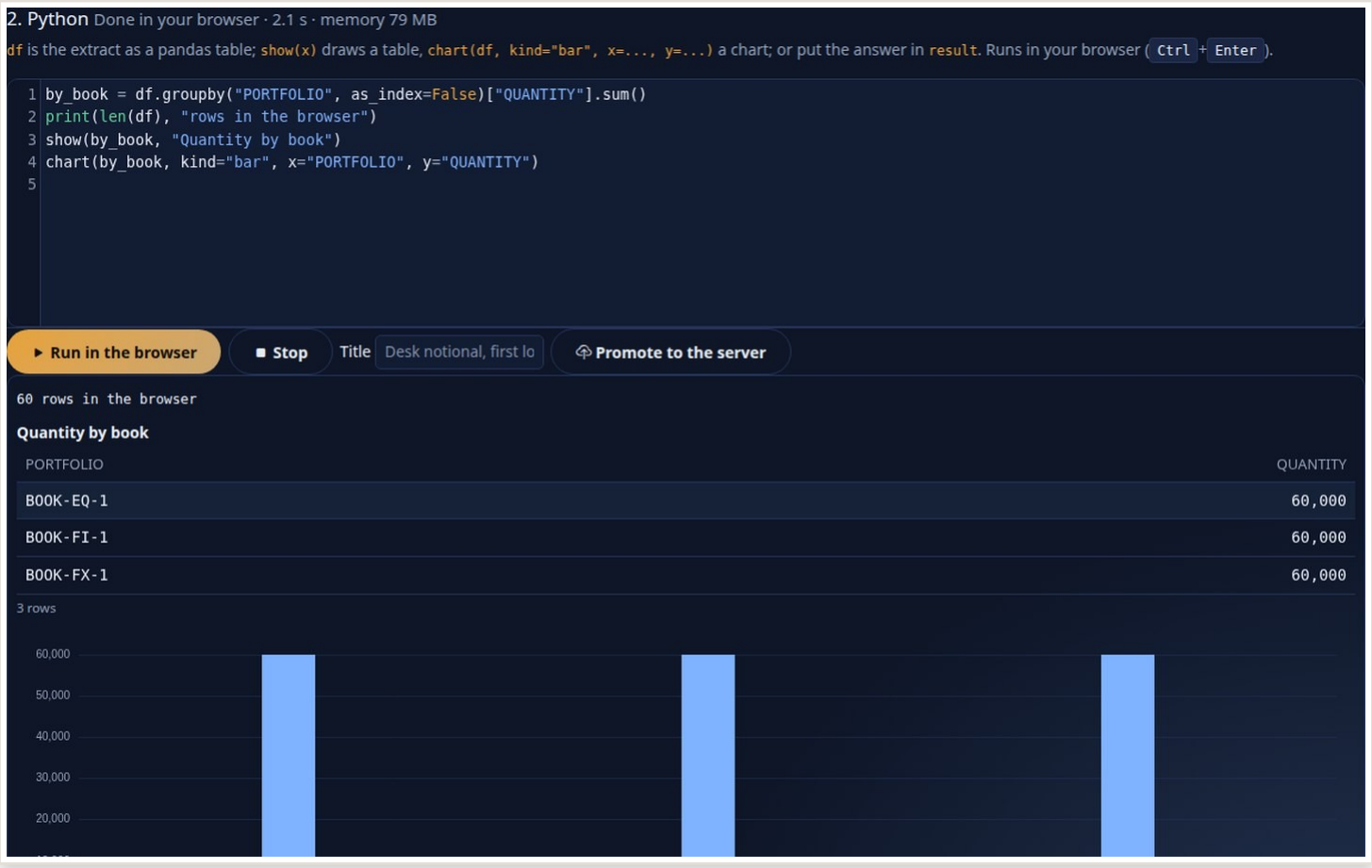
Key insight A report reader sees exactly the rows and masks they see on the terminal — BI cannot leak around them.

Reports, models, boards and themes in one studio



- **Explorer and tabs**
  Every item you may edit, by kind, with New; open items as tabs.
- **Problems and Query**
  The server checks the draft a second after each change; each visual's query and SQL.
- **A generated Format pane**
  Typed formats, conditional formatting, themes, tooltip pages, small multiples, the matrix.
- **Boards are reports**
  Board pages of live tiles beside the report's own visuals, with kiosk mode and alerts.

Python where it belongs: on the server, or on your own rows



- **Transforms, step by step**
  Inputs read through a pack's connector as you; 21 tested snippets; a profile after each step.
- **Sandboxed, warm**
  Measures, visuals and Transforms run in a fork-server child with limits.
- **Your own Python**
  BI → Python console: pandas on a masked extract, in your browser.
- **Promote to the server**
  The same code run again on the backend, compared, kept as a notebook draft.

Every heavy question is sized before it runs, then becomes a visible run

A question
visual, MQ, SQL or statement; Prefer: respond-async

The planner
sizes it from metadata only: Delta row counts, Iceberg manifests, built layers

- **Cheap** answered at once, as before
- **Heavy** a run: Runs tab, bell when it ends, share, export, schedule
- **Very heavy** asks to confirm first
- **Impossible** refused before anything runs (DRS-9029)



Chart any table
K on view tables, searches, pick lists, admin lists, history and BI tables: six types, Top N, stacking, two axes; PNG, SVG, CSV, Excel. Masked columns are never plotted.

0 connector reads to size a question (RunPlannerTest)

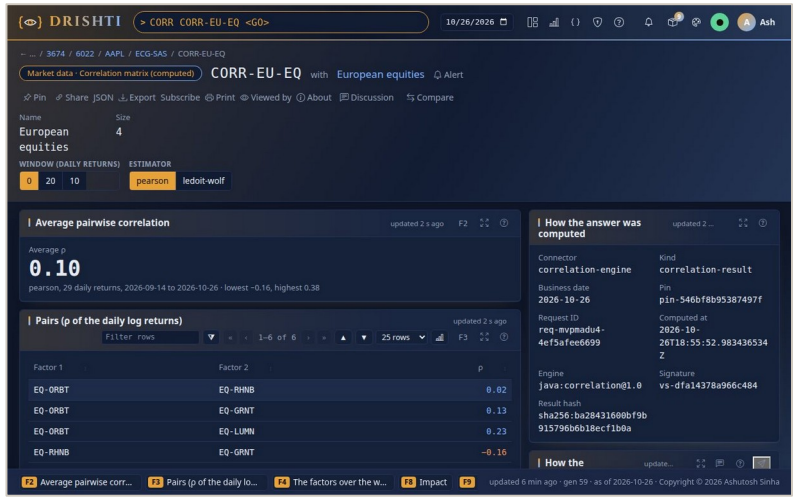
- **Fewer duplicate runs**
  An identical question joins the run already going; a finished answer is reused while the data is unchanged.
- **Masks hold**
  A run computes as its runner; a shared run is the same, re-masked or recomputed as the recipient, decided at every read.

Key insight **Refusing a year of raw trades costs a log read, not a year of builds — and an interactive slot is never held.**

Groupings and analytics computed from their members, not stored

Command	Computed from	Engine
LEIG · LEIC	GLEIF relationships: ownership groups (no member cap), entities per country	gleif-groupings
SIC	the sector's companies: count, revenue, net income	us-equities-groupings
ECG	member economies: GDP summed, growth GDP-weighted, inflation the median	macro-groupings
CORR	price history: Pearson or Ledoit-Wolf over a chosen window	CorrelationEngine
EQB	an equity's beta against its index, from both histories	BetaEngine
USEQ · IDX	ratios, yearly change of a fact, index totals: worked out by the pages	the pages

Old ids open the computed pages; a control that feeds a computed panel (CORR's estimator and window) recomputes it at once.



CORR CORR-EU-EQ: estimator and window on the page (tutorial 27).

The numbers change, and say why

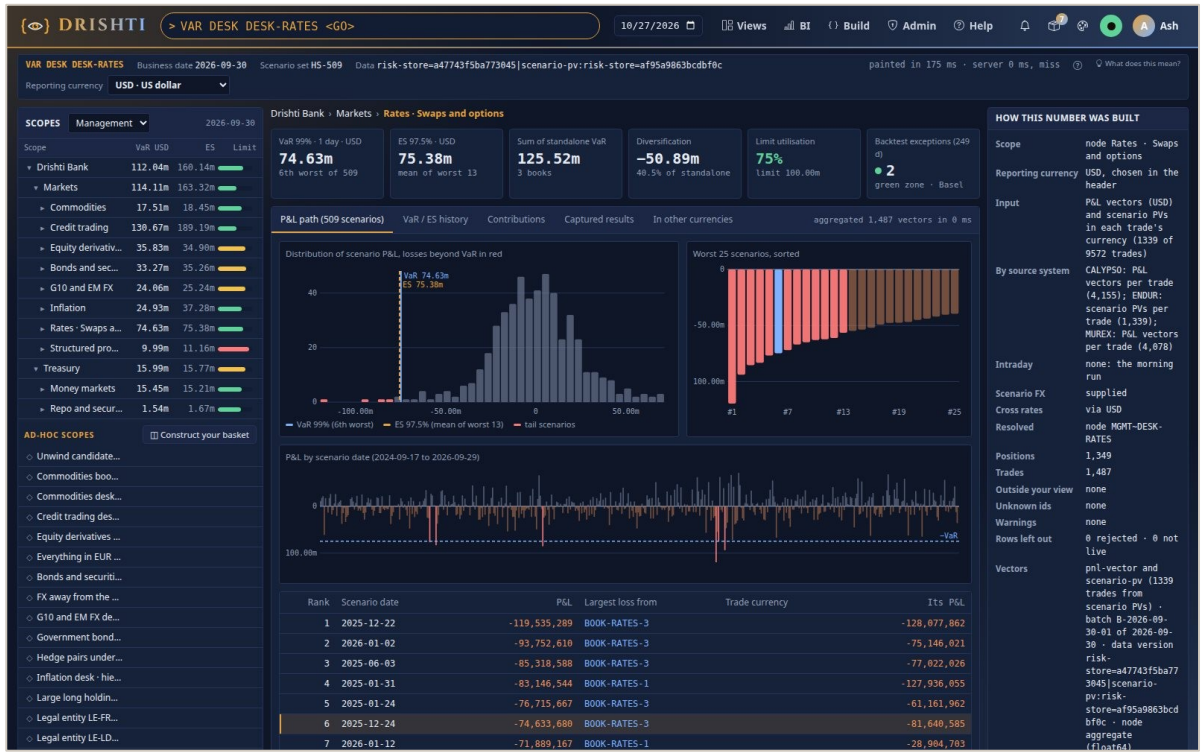
Stored correlations were set by hand at about 0.4; computed, about 0.1, because the demo's price series are independent. The reconciliation is in the guide.

Key insight **Retire the stored copy, keep the id: users follow the same links and get a figure they can re-derive.**

Risk, computed on demand

08

VaR is a computation over the trades, never a stored number



VAR DESK: the scope tree with limits, six tiles, the P&L path and its tabs (VAR_GUIDE §3).

112.04m

bank VaR 99%, 1 day, USD, 2026-09-30

160.14m

ES 97.5%: the mean of the 13 worst

509

historical scenario days, equal weights

9,572

live trades in 7,984 derived positions

131 · 28

nodes in 3 trees · portfolios

< 1 ms

a node's VaR from its aggregate

Any scope: bank, legal entity, desk, book, limit letter, position, trade, a portfolio, a rule or a basket, joined with + − &. Stored VAR retired; official records of every node of every tree, kept 7 years.

See also **In depth: the VaR engine deck — method, data flow, inputs, scale and governance.**

Scenario PVs or P&L, by trade or by position, from files or Kafka

Each system sends	By trade	By position
Scenario PVs in the trade's own currency	yes	yes
P&L vectors any currency	yes	yes

Mix and switch: systems mix on one date and may switch day to day; one form and grain per system per date (DRS-5210); a wrong record is refused and named (DRS-5208), never guessed; old files: drishti.py convert --set.

Intraday: only the changed trades, applied as new − old up the tree

Morning run
FULL: every live trade; the official records

→

Changed trades
files, or Kafka windows of 5 min (500 early)

→

Lake append
one intraday batch, latest COMPUTED_AT wins

→

Apply
book, every node above, every currency

One JSON Lines record, four forms

```
{"VALUE_TYPE":"PV","GRAIN":"TRADE",
 "TRADE_ID":"1000279","CURRENCY":"EUR",
 "BASE_PV":…,"PVS":[…509…],
 "COMPUTED_AT":"…","SOURCE_SYSTEM":"…"}
```

The guarantee
Incremental equals the same data loaded as one run, to the cent, on every node; replaying the Kafka topic after a restart writes nothing twice.

Cost of a batch
∝ trades changed × nodes above × currencies built — never the size of the book. Official numbers come only from full runs.

Key insight Source systems keep their own formats; Drishti turns every input into one internal form, a P&L vector per row.

The same trades, a different VaR per currency — exact to the cent

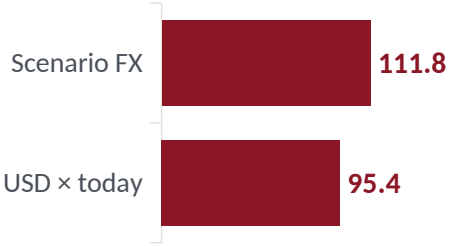


VAR DESK DESK-RATES IN EUR: every panel in EUR, the FX named.

P&L(s) = PV(s)·FX(s) − PV(base)·FX(base), every cross via USD; each node keeps its PVs summed per trade currency.

Ccy	Bank VaR 99%	Bank ES 97.5%
USD	112,041,646	160,143,809
EUR	111,837,045	137,631,546
CAD	189,801,404	230,680,413
JPY	20,883,226,318	23,578,418,999
GBP	104,139,554	125,595,958

Bank VaR, EUR m



A net 3.65bn USD[1] of value moves against the euro on the scenario days.

Setting	What it does
`drishti.var.scenario-window`	all (509) or the newest N; window=250 per request, labelled, never official
`DRISHTI_VAR_CONFIDENCE`	VaR 99% by default, 95% by setting; ES 97.5%; every label follows
`drishti.var.currencies`	USD, EUR, CAD, JPY, GBP built ahead; any other currency on first ask

Key insight A euro-based owner carries a risk a dollar-based one does not: EUR VaR is not USD VaR times today's rate.

[1] the demo bank's net value on 2026-09-30, from its positions (VAR_GUIDE.md section 15); it is why EUR VaR (111.84m) differs from USD VaR at today's rate (95.4m EUR)

Exposure computed for any scope, down to a trade, collateral path by path

Paths
every trade's MtM on every path and date (exposure-path)

→

Net
per netting set, path by path

→

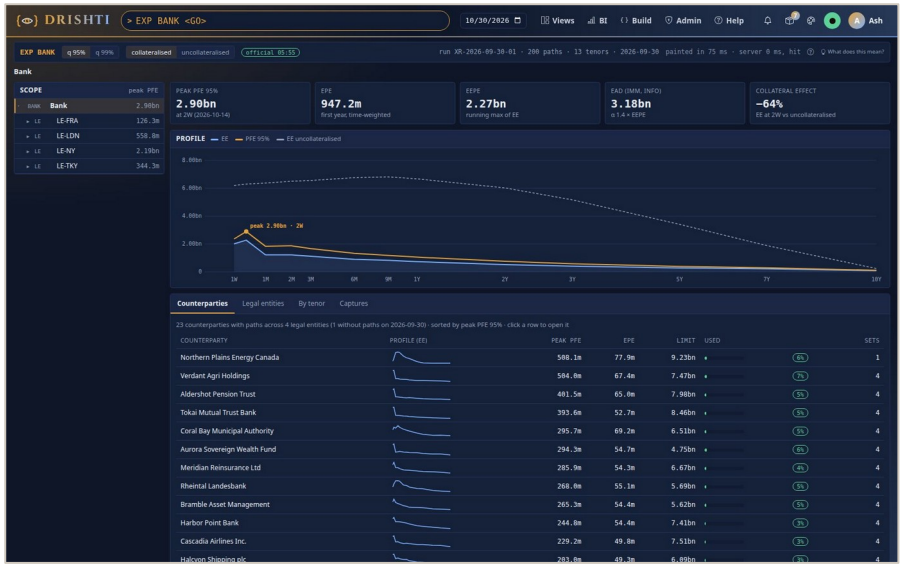
Collateral
each CSA: threshold, MTA, IA, rounding, margin period of risk

→

Measures
EE, ENE, effective EE, PFE 95/99, EPE, EEPE, peak PFE, EAD

→

Any scope
NSET, CPTY, GROUP, LE, DESK, BOOK, TRD, BANK; + − &



74
netting sets on the demo, 8,022 live trades

85.1m
peak PFE, NS-ALDERSHOT-LDN

61 ms
the bank of 9,901 sets (measured)

0.06 ms
one netting set (measured)

Explorer, pre-deal what-if (exact on the run's own paths, or a marked proxy), captures, official results, tiered retention: trade paths 20 days and month-ends; statistics 7 years.

Key insight **The stored peaks summed to 92.6bn; computed with the CSAs on the same sets, 7.6bn — the stored profile is retired.**

Derived on demand from trade-grain inputs, never stored

Trade valuations
live status, a netting set, maturity not before the date

→

Derived kinds
built once per date and data version, shared

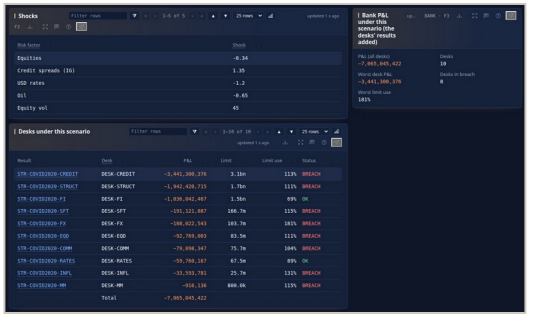
→

Beside the stored
computed, stored and their difference on the page

Command	One page per	What it computes
NSMTM	netting set	net MtM, trades, gross ±, netting benefit, net-to-gross, uncollateralised CE, by product group
CPMTM	counterparty	net MtM, its netting sets, Σ max(set net, 0) as uncollateralised CE
SSPNL	stress scenario	bank P&L (Σ desks), worst desk, breaches, limit use per desk

Live, one rule everywhere: a live-status trade past its maturity is in no total; it is counted and listed as a data-quality warning. Every total says its sum currency (USD).





Key insight **A figure on the page is a computation over the inputs, shown beside the stored copy it will replace.**

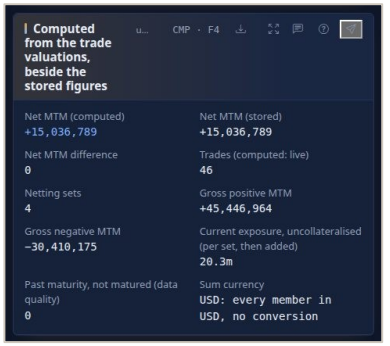
Checks on the pages; exposure on its real sources



Collateral balance: held, posted and net after haircut, from the positions



Credit limit: headroom, utilisation = used ÷ limit, and its difference



Counterparty: computed from the trade valuations, beside the stored

The exposure engine reads	Before	Now
A netting set's members and MtM	the set's stored TRADES[]	trade-valuation, live status
Collateral held today	the balance before haircut	the positions' value after haircut
Holdings of the 76 balances	6,304.4m market value	6,229.6m after haircut

BANK, 2026-09-30, USD	Before	After
Peak PFE 95%	2,858.1m	2,897.6m
EPE	945.6m	947.2m
CE	1,532.0m	1,586.8m

Key insight Every exposure figure that moved, moved for one reason: collateral now counts after its haircut.

RECONCILED AND GUARDED

55 of 76 netting sets agree at once; the other 21 are explained to the dollar

55 / 76
netting sets: derived net MtM equals the stored, at once

827
live-status trades past maturity in the other 21 (MtM 16.7m)

76 / 76
agree to the dollar with those trades added back

112,041,646
VAR BANK, USD, 2026-09-30: unchanged

Retired field	Why	Now
`netting-set.UTILISATION`	a stored ratio the page can compute	dropped; BI's card shows trades, largest net MtM
`risk-hierarchy.LIMIT_VAR`	VaR limits kept in two places	only in var-limit (same values)
`portfolio.LIMIT_VAR`	the same	var-limit, hierarchy PORTFOLIO

The guard
`tools/contracts.policy.yaml` removed: fails the build if a retired field returns.

No VaR figure moved
Trades, valuations, vectors, scenario PVs and paths are byte for byte as before.

PLANNED Retiring the stored netting-set and counterparty figures (NET_MTM, TRADES[], COLLATERAL, LIMIT) with the netting-set engine.

Key insight Each difference has a named cause; a retired field cannot come back without failing the build.

Official results other systems can read — and edits that never overwrite

Official end-of-day results: VaR and exposure

Morning full run
every node of every hierarchy, as the server

→

Captured, append-only
a restatement adds a result version; kept 7 years

→

risk-results
var-official, exposure-official, -cpty: Delta or Iceberg, a folder or S3; JSONL beside

→

BI reads them
read-only kinds VOFF and XOFC; the Market risk VaR model and report

Write-back: forms save through an edit log, with four eyes (off by default)

Submit…
the draft and a review sheet: what you saw, what is there now, your change

→

Approve
a change request; the approver is never you

→

Edit log
appended to its record's lane, acknowledged after commit; 64 lanes, epoch-fenced

→

Apply
overlay, owned value or the source through an outbox; rights re-checked

→

Recompute
reactive recomputes and live updates after each change

Conflicts are named, never overwritten: a change whose fields moved since you saw them is stopped with the three-way difference in words; DRS-9301 to 9306; Review → Changes; tutorial 24.

Key insight Engines never read the published results (a guard test): the record is for reporting, the lake stays the input.

Security, drawn as flows

09

Authentication, authorization and audit are kept apart

Authentication
Who you are

- Local passwords: PBKDF2-SHA256, 240,000 iterations
- Or SSO over OIDC: Keycloak, Entra, Okta, Ping
- Lockout after 5 failures in 15 minutes
- Disable a user and their sessions end everywhere

Authorization
What you may see and do

- Roles name the kinds you may open
- Powers: raw, author, approve, calc, layout; BI: bi, bi-author, bi-steward
- Masked fields shown as ••• without raw
- Packs assigned per user

Audit
What happened

- Sign-ins, changes, token use, purges
- Access log: who viewed what, for which date
- Hash-chained collaboration, sealed
- Legal holds and an eDiscovery export

Key insight A mask is applied where a value leaves — in views, search, emails, chat bridges and snapshots alike.

Authentication — a person signs in to the console

Browser → Console → Server

```
1. Browser → console: any page but landing, help
   → 303 to /login (or to the identity provider)
2. Password: server checks PBKDF2-SHA256 hash
   or OIDC: server verifies the ID token itself
3. Server → signed session; console sets cookie
   HttpOnly, SameSite=Lax, Secure by default
4. Every write: same-origin check (else 403)
   and JSON only (else 415)
5. Every 10 s: is this user still enabled?
   disabled, deleted or reset → session ends
```

- **Lockout**
  Five failed sign-ins lock the account for fifteen minutes; an admin's reset unlocks it.
- **The last admin is protected**
  You cannot remove the last administrator, on any node of a cluster.
- **Forced change**
  The development admin is flagged in every page until its password changes.

Key insight A cross-site form cannot post: writes must come from the console and must be JSON.

Authorization — a script acts as its owner, never as more

Scope	Opens	Still required
(none)	Read: views, search, explain, read-only commands	The owner's roles and packs
design:write · bi:write	Create and save designs and BI items	The author or bi-author power
design:approve · bi:approve	Approve someone else's design or BI item	The approve power; never your own
packs:admin	Deploy, roll back and switch packs	The admin role
loads:write	Say a batch of data landed	The pack in the owner's packs

- **Hashes only**
  Secrets are stored as SHA-256; shown once.
- **Write scopes expire**
  At most 90 days.
- **Never open**
  Users, roles, tokens, sign-in, caches and collaboration state.
- **Audited**
  Every write and every refusal.

Key insight A scope only opens the endpoint — the owner's roles are still checked on every call.

Token exchange — the host proves itself, and the user

Host app → Server → Console → Browser

```
1. Host server → POST /api/v1/embed/token  (RFC 8693)
   client: secret (hash stored) or private_key_jwt
   subject: the IdP's ID token, or a host-signed
            assertion (≤ 60 s, used once)
2. Server → ES256 token, 300 s (max 900)
   audience = app, origin-bound, scopes embed:view
3. Browser → console /embed/v1/... with the token
   console checks signature (JWKS), audience, origin
4. Server checks again: app, user, origin, kind,
   rate limit — and always masks
5. Live stream: event: token → renewed in place
```

- **Off by default**
  drishti.embed.enabled; each host app registered by an admin.
- **Always masked**
  An embedded view never shows raw values, whatever the user's roles.
- **CORS, exactly**
  Only registered origins; never *, never credentials.

Key insight The token proves who and where; the server still decides what — on every call.

Evidence, not just conversation

Chained

Every revision of a thread or share hashes the one before

Sealed

Seals cover comments, thread heads, holds and audit; verify on demand

Held

Legal holds by entity, user, thread or all (423 while held)

Exported

eDiscovery zip with a manifest; one-shot download

- **Retract and hide, never delete**

Retention keeps for ever by default; a hidden comment is still in the record.

- **Measured**

An export of 8,300 comments builds in 2.0 s and downloads (19.8 MB) in 3.1 s.

- **Only for compliance**

Legal holds and the export need the compliance power; an export downloads once.

Key insight **Tamper evidence is the honest claim: Drishti cannot stop a database administrator, but it shows when one acted.**

Evidence, by the numbers

10

HOW IT IS TESTED

Over 3,400 Java test methods, three browsers, one gate

3,400+
Java test methods (46 parameterised); the run expands them

~2,250
Console test functions, 640 of them in a browser

~640
CLI, generator and tool test functions

3
Browser engines for embedded views

- **Gates in every build**
  Error Prone and Spotless; ArchUnit for module direction; licence headers; no file over 1,500 lines; branding literals; JSONL from every generator and on every export; READMEs for every pack.
- **Pages at three widths**
  Every page at 390, 1,600 and 2,560 px; at 390 with touch: real content, 44 px targets, nothing clipped.
- **Numbers checked against an oracle**
  VaR and ES recomputed from the input files and compared with the server; intraday equals one run, to the cent; Delta equals Iceberg; a sharded cluster equals one server.

Key insight **Any failing test in any of the three suites stops the push — on Java 21 and on Java 25.**

DOCUMENTATION AS EVIDENCE

Retaking 351 pictures on the current UI found five bugs — all fixed

351
pictures under docs/ retaken on the current UI by tools/docs/screenshots.py

29
hands-on tutorials, from the first 15 minutes to VaR, exposure, funds and Iceberg

2 + 2
documents and Help pages per measure: a guide and a tutorial

193
guides, references and tutorials, plus 113 contract pages

Found by the screenshot run	Fixed
A new pack's job reached through another console worker answered DRS-5006 no such job	Job ids name their worker; the call is relayed to it
An installed pack's help guide never reached the console ('There is no guide called helpdesk-pack')	The console reads the server's installed packs first
The About drawer listed the chosen Sutra under 'Not chosen'	Listed once, as chosen
A Sutra control's open multi-select pushed the header's controls apart	The list floats over the panel
The BI test stack's sample dataset failed the check, DRS-9010	Rewritten in today's grammar

Key insight Documentation is part of the release: a picture that cannot be retaken is a bug report.

VaR and exposure from Iceberg equal Delta, to the cent

Test	What it proves	Latest run
`VarFormatEqualityTest`	the same lines in Delta and Iceberg: VaR, ES, component and incremental VaR, worst scenarios, limits and limit letters, USD and EUR; intraday batches; official records	green
`ExposureFormatEqualityTest`	EE, PFE, EPE, profiles, contributions and official records, both formats	green
`LakeAppendsTest`	a Kafka window lands on an Iceberg risk lake as one append	green
`OfficialSinkTest` · `OfficialS3Test`	official tables in Delta and Iceberg, a folder or S3; six writers at once; a Delta commit on S3 is an exclusive create	green
`OfficialReadersTest`	BI's readers follow the official format by themselves	green

NOT RUN The Glue and S3 Tables runs on AWS (GlueLiveTest, check-glue.sh): written, never run; they need an AWS account.

On the demo data

Figure	Delta	Iceberg
VAR BANK, 2026-09-30 (USD)	112,041,646	112,041,646
Risk tables moved	—	14 tables, 955,124 rows, 579 s

A server on the Iceberg lake answered VAR BANK, the limit letter LL:LL-0003 and XPO BANK and published its official records.

Demo figures from Drishti's synthetic data, one workstation: an observation, not a sizing rule. Delta stays the default; Iceberg is chosen by configuration.

Key insight One connector switched from plugin: delta to plugin: iceberg, the same rows, the same numbers.

Trees rebuilt from columns; upgrades that keep every edit

`HierarchyEquivalenceTest`

MGMT and LEGAL rebuilt from columns

- The same node ids, parents, levels and members as the risk-hierarchy rows
- Aggregates equal to the bit; VaR, ES, components and worst scenarios to the cent, USD and EUR
- Limits equal; every limit letter the sum of its rows

Latest run: 3 tests green

`HierarchyUpgradeWithoutLettersTest`

Data written before the letters

- P&L vectors and exposure paths without LIMIT_LETTER
- Only LL is skipped, with a DRS-5211 warning and its fix
- VaR, exposure, MGMT, LEGAL and the official captures answer as before

Latest run: 1 test green

`ConnectorUpgradeTest`

A server on the previous release's files

- Every unedited file equals the current template
- The edits stay (an edited risk-store and var-whatif); missing keys added
- var-result, var-whatif with ccy and exposure-result answer

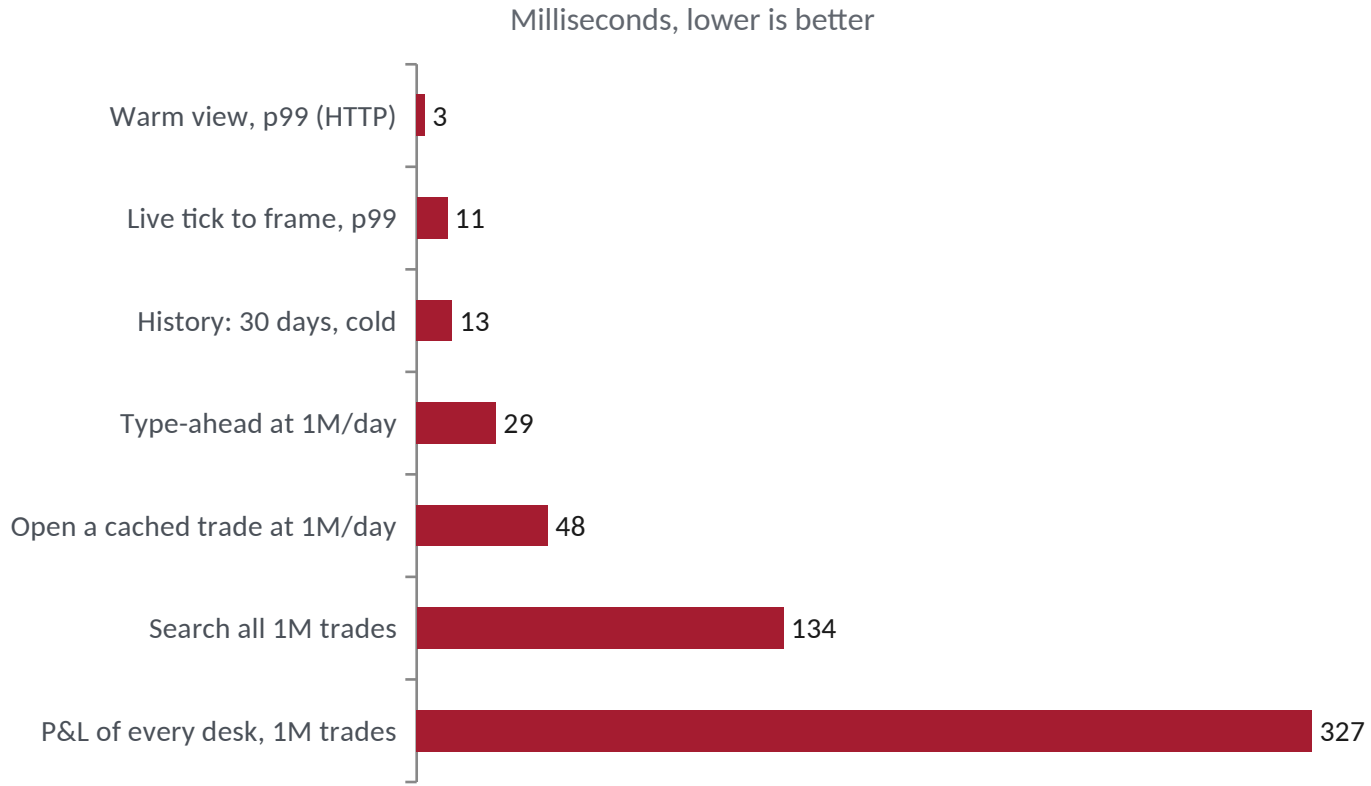
Latest run: 3 tests green

ColumnHierarchyBenchmark: 10,000 trades, 700 limit letters: the trees build in 65–99 ms against 53–197 ms without them; the VaR of all 1,412 nodes takes about 30 ms.

Key insight A new tree is configuration; an upgrade never loses an edit or a date.

PERFORMANCE · MEASURED

Fast where people wait



Gate	Target	Measured
Warm view p99	< 50 ms	3.0 ms
Live p99	< 40 ms	11.2 ms
History, 30 days	—	13 ms cold, < 1 ms warm

- **One workstation**
  24 cores; 1,000,000 trades a day for three days, 5.2 GB.
- **Cold vs cached**
  Opening a trade cold is 137–157 ms; from the cache 48 ms.

Key insight **The gates are written down first and measured after — and every one has headroom.**

SCALE · 35.7 MILLION TRADES

Tens of millions of trades, fifty users, no failures

35.7M
Trades: ENDUR, CALYPSO, MUREX, 30 days, loaded while serving

179/s
Requests a second, 50 users against the server, 0 failures

0.21 s
Form search p50, from 5.8 s before the form was kept per data version

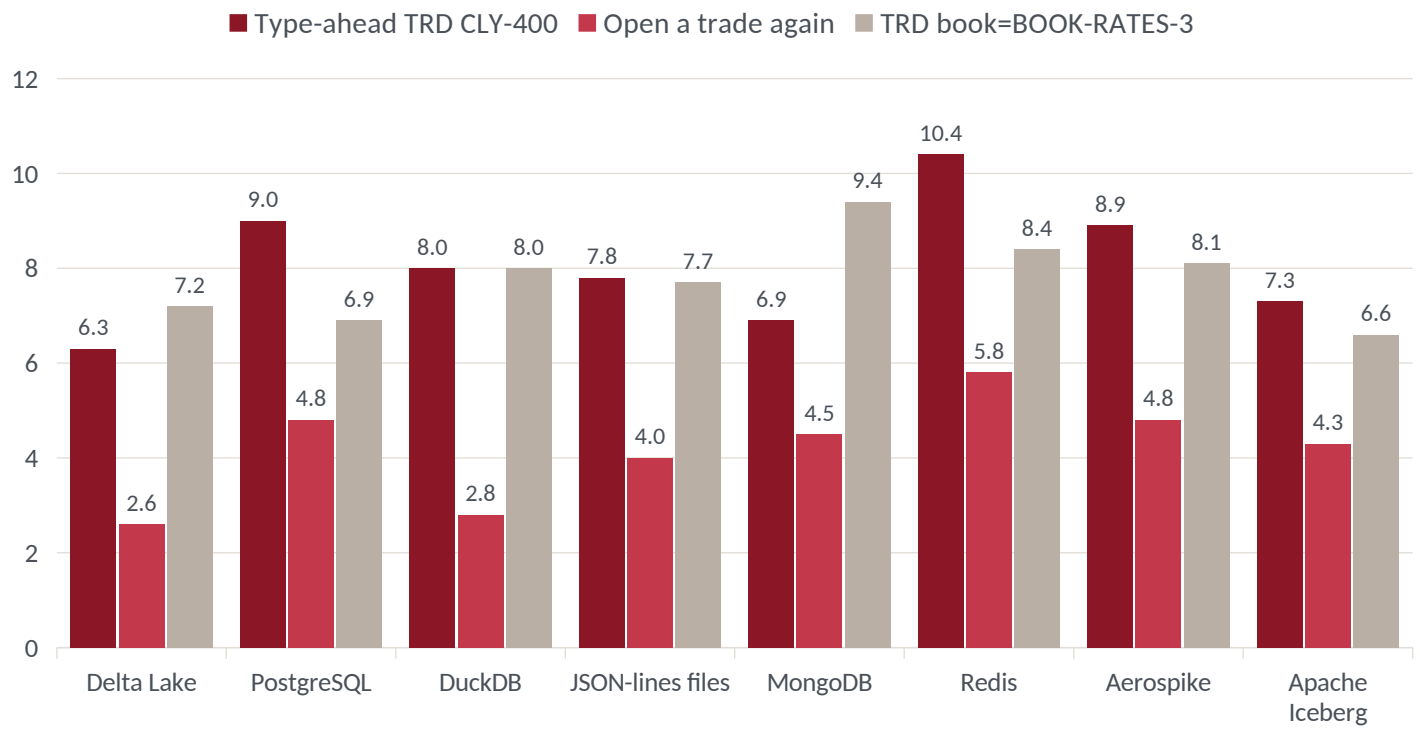
182/s
Console pages a second, 50 users, 0 failures

Data	Measure	Users	Result
35.7M trades	Single trade / grouped page, p50 / p95	50	0.16 / 0.73 s, 0.17 / 0.73 s
35.7M trades	Server heap while serving	50	3.7 GB of 8 GB
400,000 restated	A day reloaded, 10 % amended, while serving	30	2 min 54 s, 202 requests/s, 0 failures
8.2M trades	Single trade / grouped page from disk, p50	50	18 ms / 70 ms

Key insight One workstation, an 8 GB heap: the figures and their method are in PERFORMANCE.md and SCALE_STRATEGY.md.

The same book in eight stores, the same questions over HTTP

Median milliseconds at 50,000 trades a day (lower is better)



Store	Size	Load
Delta Lake	270 MB	10.2 s
PostgreSQL	739 MB	22.3 s
DuckDB	122 MB	22.3 s
JSON-lines	1,322 MB	15.4 s
MongoDB	202 MB	17.1 s
Redis	183 MB	20.2 s
Aerospike	1,176 MB	18.8 s
Iceberg	133 MB	26.4 s

Measured on an earlier release, 2026-10-01, one workstation; not re-measured since. Four stores were also measured at a million trades a day.

Key insight Every store answers an interactive question in single-digit milliseconds at this size; the choice is about operations.

VaR and exposure answers in milliseconds, at 100,000 trades in memory

VaR (VarScaleBenchmark, VarTierBenchmark)

Measured	Time	On
A node's VaR and ES	0.48 ms	tiered store, 100k trades
The bank from float32 rows	3.4 ms	tiered store, 100k trades
The bank from float64 rows	58 ms	heap tier, 1M trades
A basket of 50,000 positions	2.6 ms + ≈ 45 ms ids	tiered store, 100k
First load of a date	0.8–1.2 s	tiered store, 100k
Loading the demo date	572 ms	9,572 trades, the server

Exposure (ExposureTierBenchmark)

Measured, 100k trades, 9,901 sets	Time
Load a date: rows into value matrices	785 ms
Collateral, E, N, hot file written	1,250 ms
Reopen the date from its hot file	157 ms
Scope BANK (every set)	61 ms
Scope CPTY: (20 sets) · NSET: (1 set)	0.22 · 0.06 ms
Pre-deal exact, Java · proxy	0.11 · 0.007 ms

Why it is fast

Vectors add: a node's vector is summed once and kept; a hot file per date is memory-mapped (float32 rows, float64 sums); sums run on every core in a fixed order — the same bits every time.

What is extrapolated, and said so

1M trades × 50 dates (VaR) and a 1M-trade, 100,000-set bank (exposure) are extrapolated from these runs[1]; a workstation run stays at 100,000 trades in memory.

Key insight **Measured on one workstation (24 threads, 61 GB, JDK 25, NVMe); the full million-trade runs are for a real server.**

[1] linear in trades (× 10 from 100,000 to 1M) and in dates from the measured runs above (VAR_GUIDE.md 27.3, EXPOSURE_GUIDE.md 27.4); not measured, to be confirmed on a server

Written to break a promise — and fixed

94
Findings in round one (10 high), security, data and UX

72
Findings in round two, the build workbench (0 high)

~100
Findings fixed in a six-hour adversarial QA, five of them P1

4,640
Randomised searches matched brute force exactly

Round	Scope	High	Outcome
2026-10-01	Security, data and UX	10	All high fixed, each with its commit
2026-10-03	Build workbench: 25 endpoints × 6 callers	0	All 18 medium fixed
2026-10-05	Data safety and behaviour (two rounds)	2	All resolved or owned
2026-10-20	Two testers, four rounds, four soaks	5 P1	All P1 fixed; SOAK4-01 open
2026-10-21	BI runs: planner, sharing, adoption	—	Masks hold both ways

Key insight **No XSS anywhere, about 250 settings that match the code, 190 grammar fuzz cases with clean problem codes.**

Where a number changed, the reason is written down

Figure	Stored said	Computed says	Why
Commodities desk VaR	446.9m	17.5m	the stored figure came from a formula with no netting, not historical simulation
Exposure peaks, 76 sets	92.6bn	7.6bn	the generator scaled peaks with gross notional and ignored the CSAs
Correlations	≈ 0.4, set by hand	≈ 0.1	the demo's price series are independent
DIA active share	57.6%	1.8%	members from DIA's SEC holdings; the Dow weighed by price
QQQ active share	26.75%	11.2%	Nasdaq-100 members from QQQ; capped as the index does
Firm-wide VaR limit	210m (legal tree)	150m	one limit on both trees' roots; every VaR unchanged
Economy debt	a 2031 projection	latest actual year	found while writing the Knowledge pages
Joins between kinds	assumed	11 do not hold	DATAMAP checks every join against the data

Key insight Each change is reconciled in its guide, item by item: the evidence is the reason, not just the new number.

Nothing reaches main without the full gate

Licences
A header on every file

→

Java 21
Full verify; bytecode pinned to 21

→

Java 25
Verify in a copy of the commit

→

Console
Tests on parallel workers

→

Tools
CLI tests

→

Push
develop, then main

~1 hour
A full drill: Java 21 verify, then Java 25, console and tools side by side

45
Tagged releases, 30 September to 29 October 2026

2,471
Commits, every push through the drill

Key insight A failed test stops the push; a flaky one is retried once and written down — never ignored.

Operating it

11

From a clone to a live view in about fifteen minutes

Seven commands

```
./mvnw -q package -DskipTests -pl drishti-server -am
uv venv --python 3.13 drishti-console/.venv
uv pip install --python drishti-console/.venv/bin/python \
    -r drishti-console/requirements.txt
python tools/drishti.py lake demo
java -Djava.io.tmpdir=data/tmp/server -Xmx4g \
    -jar drishti-server/target/drishti-server-<v>-exec.jar
python drishti-console/run_drishti_web.py
# open :17480/t → TRD MX-20000001 <GO>
#                 VAR BANK <GO>   EXP BANK <GO>
```

GETTING_STARTED

About 15 minutes: build, the demo lakes, a view

IDE_GUIDE

Server in IntelliJ, console in PyCharm

CLI_GUIDE

pack make, history, var, exposure, convert

USER_GUIDE

Every page, every key; 29 tutorials beside it

Key insight Start with your own JSON Lines: one pack make, one deploy, and your records have a terminal.

From a folder of JSON Lines to a deployed pack

One command

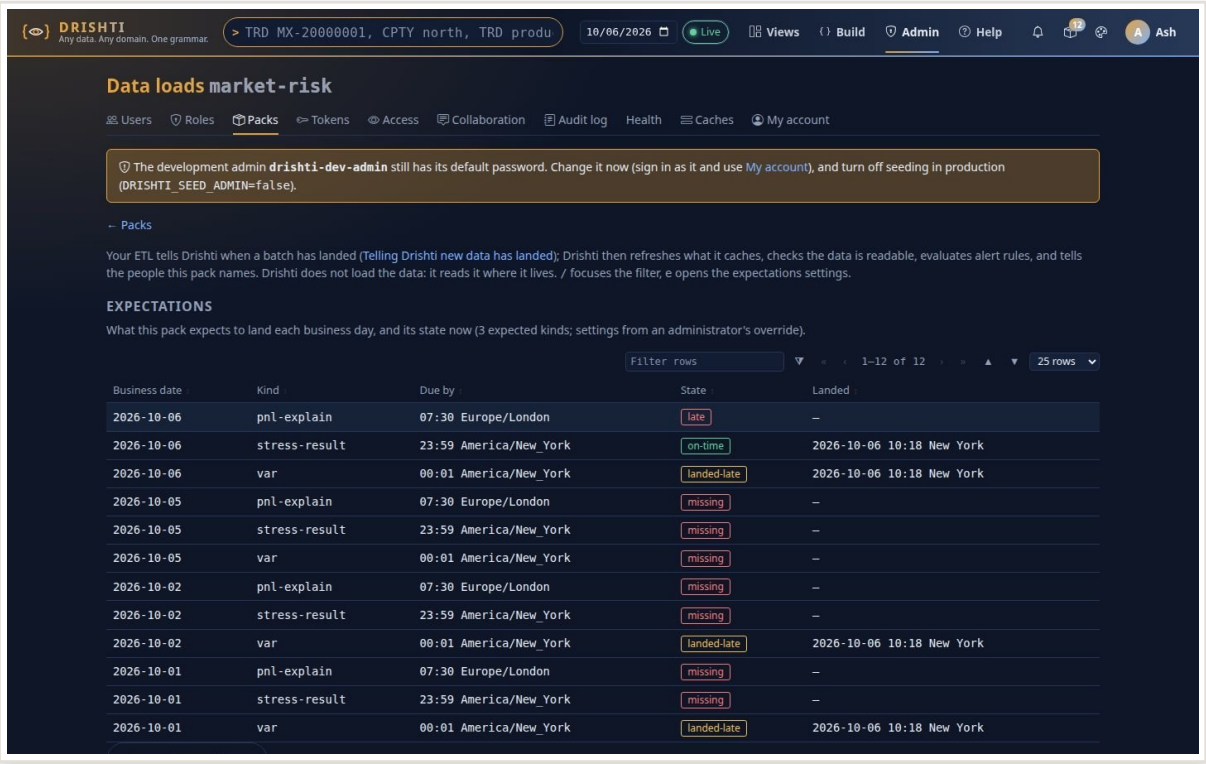
```
drishti.py pack make ./trades --pack desk \
    --kind trade --key TRADE_ID \
    --match PRODUCT_TYPE --date TRADE_DATE
# → desk/ pack, sutras per product type,
#   Delta lake by date, desk.tar.gz + sha256,
#   README.txt, run scripts
```



- **Checked before it lands**
  Checksums, paths, server version, Sutra tests and an optional Ed25519 signature.
- **Never stuck**
  The previous version is kept; a failed deploy rolls back; history has Roll back.

Key insight **Archives carry the pack, never the data — where data lives stays an admin's decision.**

Tell Drishti new data has landed — and it acts on it




```
drishti.py data landed --pack trading \
    --kind trade --date 2026-10-05 --rows 10000
# or POST /api/v1/packs/trading/loads
```

- **Refresh, verify, alert, notify**
  Caches refreshed, the date checked, alert rules run on it, bell, inbox and email.
- **Late is a fact, said once**
  Per-kind expectations with time zone and calendar; Health shows 'data late'.
- **Safe to repeat**
  Idempotent per pack, kind, date and batch; token scope loads:write.

Every build step from the command line

Group	What it does	Examples
sutra · pack	Lint, test, preview Sutras; make, sign, deploy packs	sutra test, pack make, deploy, rollback
data · lake · convert	Ingest JSONL, build lakes, turn CSV into JSONL	landed, lake demo --names, convert
server · connector · net	Packs and commands on or off, connectors, members	packs off, commands off, net members
bi · design · view	Models, SQL, runs, statements; designs; a view	bi sql --format jsonl, bi runs
history · contract	An entity over days; every command's contract	history TRD 1000015 30, contract check
var · exposure · compute	The risk engines as you see them; reactive kinds	var show … --ccy EUR, exposure profile
change · edits · doctor	Write-back changes, Admin → Edits, a setup check	change submit, edits status, doctor

Key insight Anything the console can build, a script can build — and a pipeline can check it.

Where it is going

12

What is built, planned and parked

Item	What it is	Status in the documents
Today	VaR and exposure computed for any scope, sharded across servers; Delta or Iceberg; the parallel loader; three editions and lenses; computed attributes; contracts, BI, Sutra Next, history	
Risk next	Stressed and 10-day VaR with ES backtests; the 200K-position benchmark (one server against a sharded cluster)	Planned
Screens next	Arrow from server to browser for large tables and charts; a client-side renderer of the ViewModel	Planned
Lakes next	Delta ↔ Iceberg conversion; in-place Delta schema evolution; plain tables and Unity Catalog; the Glue and S3 Tables live runs	Planned; Glue not run
Trading and market risk	Valuation totals, stress, FRTB, SIMM and P&L explain computed for any scope	Parked
Counterparty, balance sheet	Counterparty risk (SA-CCR, CVA, collateral, margin); balance sheet (large exposures, LCR, NSFR, IRRBB, IFRS 9)	Parked
AI and GenUI	The Sajha MCP connector, AI management, screens composed from UI-kit docstrings	Parked until the balance-sheet measures

Appendix: reference



248 error codes, every one documented

```
$ curl -s server/api/v1/views/trade/NOPE-404
Content-Type: application/problem+json
{ "status": 404,
  "code":   "DRS-1001",
  "title":  "entity not found",
  "detail": "no source holds the entity" }
```

Family	Area	Codes
DRS-1xxx	Entities and sources	14
DRS-2/3xxx	Grammar, Sutras, inference	11
DRS-4xxx	Commands and search	19
DRS-5xxx	Requests, Drishti*net, VaR, exposure	51
DRS-6xxx	Users, roles, tokens	11
DRS-7xxx	Collaboration	16
DRS-8xxx	Embedding	6
DRS-9xxx	Business intelligence, runs	120

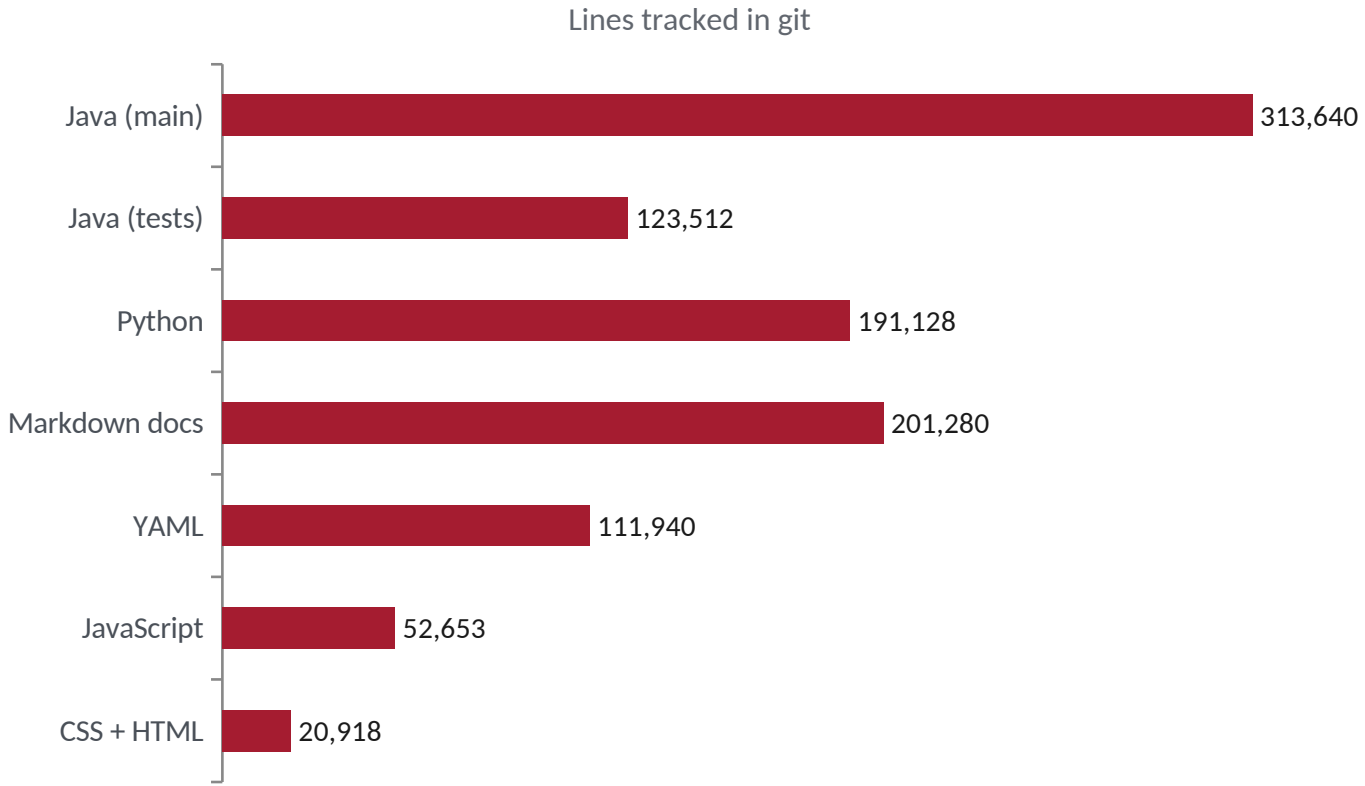
248
Codes in nine families, one table

47
Sutra problem codes in the grammar reference

RFC 7807
problem+json, the same shape everywhere

Key insight **Errors are a contract: a stable code, an HTTP status and a row that says what to do.**

Where the work is



- **193 guides and references**
  User, admin, CLI, BI, packs, connectors, Knowledge, 29 tutorials and 22 ADRs; 113 generated contract pages; two documents per measure.
- **Every screen documented**
  Over 800 pictures under docs/, taken by scripts on the current UI over the demo lakes.
- **21 modules, 19 plugins**
  Plus a server, integration tests and benchmarks; the risk engines in drishti-compute.

REFERENCES

Related documents

Document	What it holds
README.md · docs/guides/GETTING_STARTED.md	the name and the tagline; from a clone to a live view in about 15 minutes
docs/guides/USER_GUIDE.md · CLI_GUIDE.md	every page and key; the command line
docs/architecture/IMPLEMENTATION_PLAN.md	what is built, planned and parked; known gaps
docs/architecture/EDITIONS.md · docs/admin/EDITIONS_GUIDE.md	Drishti Risk, World and Market; editions and lenses
docs/guides/DATA_FORMATS.md · docs/guides/COMMAND_CATALOG.md	JSON Lines as the data standard; a contract per command
docs/guides/VAR_GUIDE.md · EXPOSURE_GUIDE.md	risk computed on demand: VaR and exposure
docs/architecture/COMPUTED_ATTRIBUTES.md	the attributes on the pages, derived from trade-grain inputs
docs/admin/PERFORMANCE.md · docs/architecture/SCALE_STRATEGY.md	the measured numbers and how they were taken
RELEASE_NOTES.md · CHANGELOG.md	the release history (this deck is version-free)
docs/knowledge	the ideas behind the data
docs/publications/Drishti-VaR-Engine.pptx	the VaR engine in depth
docs/publications/Drishti-on-AWS.pptx	Drishti on AWS: the lake, Delta or Iceberg



Thank you

Any data. Any domain. One grammar.

Drishti • Ashutosh Sinha • October 2026